

修士論文

バードストライク検出に向けた
コンピュータビジョンと
UAV の活用

山形大学大学院
理工学研究科数理科学分野
23321427 千葉歩夢

2025 年 2 月

Abstract

ここに概要を書きます。

Contents

1.1	はじめに	2
1.1.1	背景と目的	2
1.1.2	先行研究の概要	2
1.1.3	引き継ぎの理由と目標	2
1.2	方法とアルゴリズム	3
1.2.1	科学用語の説明	3
1.2.2	検出方法	5
1.2.3	AutoEncoder の改良	6
1.2.4	モデルのアップデート方法	10
1.3	使用したデータの説明	10
1.3.1	プログラミング環境と使用したライブラリ	11
1.3.2	想定環境	12
1.3.3	画像データ	12
1.3.4	モデルデータ	13
1.4	実験	14
1.4.1	物体検出の試み	14
1.4.2	モデルアップデートの有用性	15
1.4.3	背景画像の分類	16
1.4.4	物体画像の抽出	22
1.4.5	モデルのアップデート	26
1.4.6	物体検出の試み	33
1.4.7	物体検出の仕組み	34
1.4.8	出力画像の精度向上	41

1.1 はじめに

1.1.1 背景と目的

近年、再生可能エネルギーの 1 つである風力発電の導入が注目されているが、発電機の建設には問題点も多い。バードストライクはその問題点の 1 つであり、回転する風車のブレードに鳥類が衝突する現象である。山形県企業局では、当局が管理している風力発電機に対してバードストライク調査を行っているが、仕事量の多さや正確性に幅があることが懸念されている。この懸念を解消するために、ドローンを用いて発電機周辺の土地を撮影し、自動的に鳥の死骸を発見できるプログラムを作成することが本研究の目的である。

詳しい内容は昨年度の論文で詳細に説明しているので、そちらを参照して欲しい。

1.1.2 先行研究の概要

昨年度、山形大学の先輩である梅津魁秀氏は、深層学習を用いたバードストライク調査プログラムの基礎的な研究を行った。彼の研究では、AutoEncoder を用いて物体検出モデルを作成し、ドローン画像から鳥の死骸を検出する手法を提案した。具体的には、ドローン画像を「物体が存在する画像」と「物体が存在しない画像」に分類し、「物体が存在する画像」と GUI を組み合わせることで手動で鳥の死骸を検出する方法を開発した。彼の研究により、物体画像に関して F2-score が 0.83 という高い精度で物体の有無を判定することができた。

1.1.3 引き継ぎの理由と目標

梅津氏の研究は有望な結果を示したが、実用化にはさらなる改良が必要である。特に、写りゆく四季に適応した汎用的なモデルを構築することと、物体画像から鳥の死骸の画像を自動で検出できるようにすることが挙げられる。本研究では、これらの課題に対処し、より実用的なバードストライク検出システムの開発を目指す。

1.2 方法とアルゴリズム

ここからはバードストライク調査プログラムの作成に使用する方法やアルゴリズムを紹介する。なお、今年度は引継ぎのため、重複する部分は省略し、新規の内容や変更した内容のみ説明する。

1.2.1 科学用語の説明

本研究では、目標達成のために、さまざまな技術を使用している。以下に、研究の中で使用される主要な科学用語を説明する。

AutoEncoder (オートエンコーダ)

AutoEncoder は、入力データを圧縮し、その圧縮されたデータから元のデータを再構築することを目的としたニューラルネットワークの一種である。主にデータの次元削減や特徴抽出に使用される。AutoEncoder はエンコーダとデコーダの 2 つの部分から構成され、エンコーダは入力データを低次元の潜在空間にマッピングし、デコーダはその潜在空間から元のデータを再構築する。

CLIP (Contrastive Language-Image Pre-Training)

CLIP は、OpenAI によって開発されたモデルで、画像とテキストのペアを用いて学習することで、画像とテキストの間の関連性を高精度で捉えることができる。CLIP は、画像エンコーダとテキストエンコーダを用いて、それぞれの入力を高次元のベクトルに変換し、これらのベクトル間のコサイン類似度を最大化するように学習する。これにより、画像とテキストの間の関連性を高精度で捉えることができる。

ResNet (Residual Networks)

ResNet は、深層ニューラルネットワークの一種であり、深いネットワークにおける勾配消失問題を解決するために開発された。ResNet は、残差ブロックを導入することで、非常に深いネットワークでも効果的に学習を行うことができる。これにより、画像認識の精度が大幅に向上することが示されている [1]。

RN50 および RN50x64

RN50 は、ResNet-50 をベースとしたモデルであり、50 層の深層ニューラルネットワークを用いて画像をエンコードする。RN50x64 は、ResNet-50 を 64 倍に拡張したモデルであり、より高精度な画像特徴量を抽出することができる。RN50x64 は RN50 の各層のフィルタ数を 64 倍に増やすことで、より多くの特徴を捉えることができる。これにより、より高精度な画像認識が可能となる。これらのモデルは、CLIP の画像エンコーダとして機能し、テキストエンコーダと組み合わせることで、画像とテキストの関連性を高精度で捉えることができる。

主成分分析 (Principal Component Analysis, PCA)

主成分分析は、高次元データを低次元に変換するための統計手法である。データの分散を最大化する方向に新しい軸（主成分）を見つけ、その軸にデータを投影することで次元削減を行う。これにより、データの重要な特徴を保持しつつ、次元を削減することができる。各主成分の寄与度は、その主成分がデータの分散をどれだけ説明して

いるかを示す指標である。寄与度が高い主成分ほど、データの重要な情報を多く含んでいることを意味する。通常、最初の数個の主成分がデータの大部分の分散を説明するため、次元削減後のデータ解析において重要な役割を果たす。

K-means クラスタリング

K-means クラスタリングは、データを K 個のクラスに分割するための非階層型クラスタリング手法である。各クラスは、クラスを中心（重心）からの距離が最小になるようにデータポイントを割り当てることで形成される。K-means クラスタリングは、データのパターンを見つけるために広く使用される。

コサイン類似度

コサイン類似度は、2 つのベクトル間の類似度を測るための指標である。ベクトル間の角度のコサイン値を計算することで、ベクトルの方向がどれだけ似ているかを評価する。コサイン類似度は、0 から 1 の範囲で値を取り、1 に近いほどベクトルが類似していることを示す。

シード (Seed)

シードは、乱数生成の初期値を設定するための値である。シードを固定することで、実験の再現性を確保することができる。

ゼロショット学習 (Zero-Shot Learning)

ゼロショット学習は、トレーニング時に見たことのないクラスのデータを分類する能力を持つ機械学習手法である。ゼロショット学習では、事前に学習した知識を利用して、新しいクラスのデータを認識する。これにより、少ないデータで新しいクラスを分類することが可能となる。

F2 スコア

F2 スコアは、精度 (Precision) と再現率 (Recall) の調和平均を計算するための指標であり、再現率をより重視する場合に使用される。F2 スコアは以下の式で計算される：

$$F2 = \frac{(1 + 2^2) \cdot (\text{Precision} \cdot \text{Recall})}{(2^2 \cdot \text{Precision}) + \text{Recall}}$$

F2 スコアは 0 から 1 の範囲で値を取り、1 に近いほどモデルの性能が高いことを示す。F2 スコアが高い場合、モデルは高い再現率を維持しつつ、精度も高いことを意味する。逆に、F2 スコアが低い場合、モデルは再現率または精度のいずれか、または両方が低いことを示す。F2 スコアは、特に不均衡データセットにおいて、モデルの性能を評価するために有用である。

1.2.2 検出方法

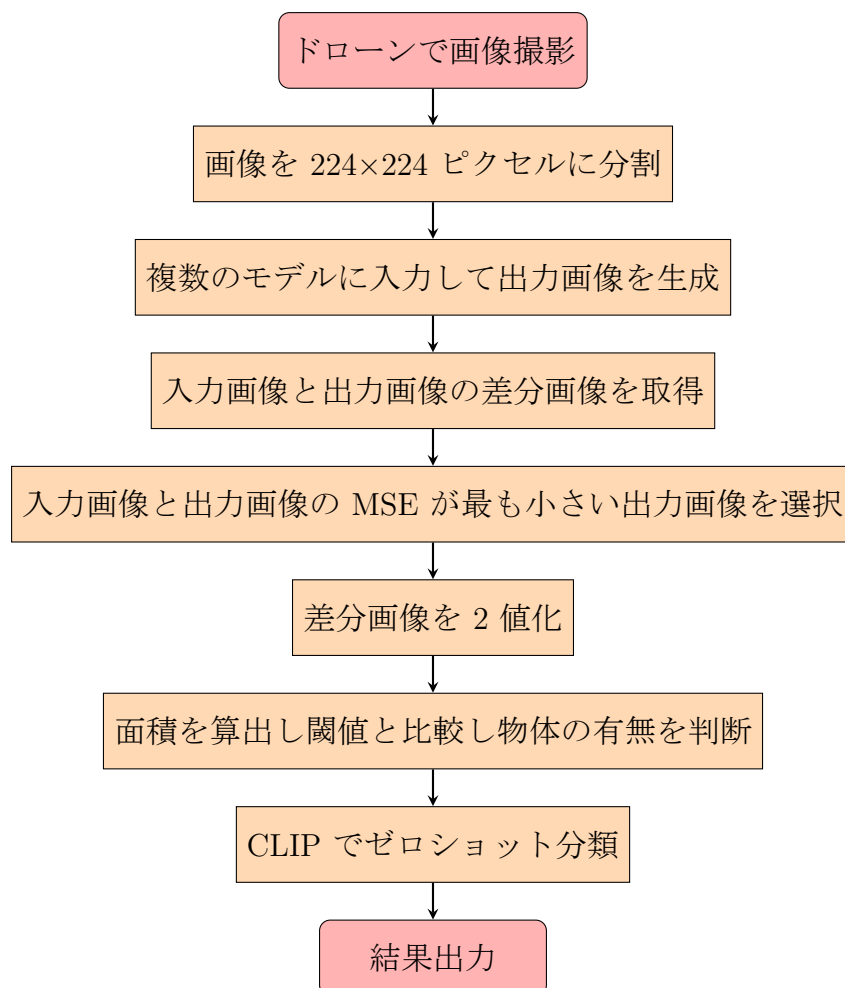


Figure 1.1: バードストライク検出システムの流れ

本研究では、ドローン画像からコンピュータビジョンを用いてバードストライク検出システムを製作した。具体的な流れは以下の通りである。

始めに、ドローンで探索範囲を撮影し、画像（3000×4000 ピクセルが複数枚）を準備する。次に、それらの画像を 224×224 ピクセルに分割し、背景の種類が複数あるため、複数のモデルに入力する。

出力画像と入力画像の差分画像を取ることで、物体部分だけが浮き彫りになった画像を得る。複数のモデルを使用しているため、モデル分の差分画像が生成される。最も適したモデルによる差分画像を選択するために、入力画像と出力画像の平均二乗誤差（MSE）の値が最も小さい画像を採用する。

採用した差分画像に関して、2 値化し、面積を算出して閾値と比較することで物体の有無を判断する。抽出できた物体画像に関しては、その画像が鳥の画像なのかを判定する必要があるが、現状では鳥の死骸の画像が少ないため、教師あり学習は困難である。そこで、ゼロショットで分類できる CLIP の導入を検討している。

ここまでは内部的な処理の話であり、実務での使用のためにさくらの VPS の導入や Django でのアプリケーション開発を学部生二人が行った。これについては巻末に付録として記載する。

1.2.3 AutoEncoder の改良

本研究では、以前の AutoEncoder モデルと比較して、構造をシンプルにすることで性能向上を図った。以下に、以前のモデルと現在のモデルの違い、およびその違いによる改善点について説明する。

モデル用語の説明

- **Conv2d** (入力, 出力): 2 次元畳み込み層。入力チャンネル数と出力チャンネル数を指定し、画像の特徴を抽出するために使用される。例えば、Conv2d (3, 16) は 3 チャンネルの入力画像を 16 チャンネルの出力に変換する。
- **BatchNorm2d**: バッチ正規化層。ミニバッチごとにデータを正規化し、学習の安定性と速度を向上させる。内部共変量シフトを減少させる効果がある。
- **ReLU**: Rectified Linear Unit (ReLU) 活性化関数。入力が 0 以下の場合は 0 を出力し、0 より大きい場合はそのまま出力する。非線形性を導入し、モデルの表現力を高める。
- **MaxPool2d**: 最大プーリング層。指定されたウィンドウサイズ内で最大値を抽出し、空間的なサイズを縮小する。特徴マップのダウンサンプリングに使用される。
- **ConvTranspose2d**: 2 次元逆畳み込み層。畳み込み層の逆操作を行い、特徴マップの空間的なサイズを拡大する。アップサンプリングに使用される。

以前の **AutoEncoder** モデル

以前の AutoEncoder モデルは、以下のような構造を持っていた。

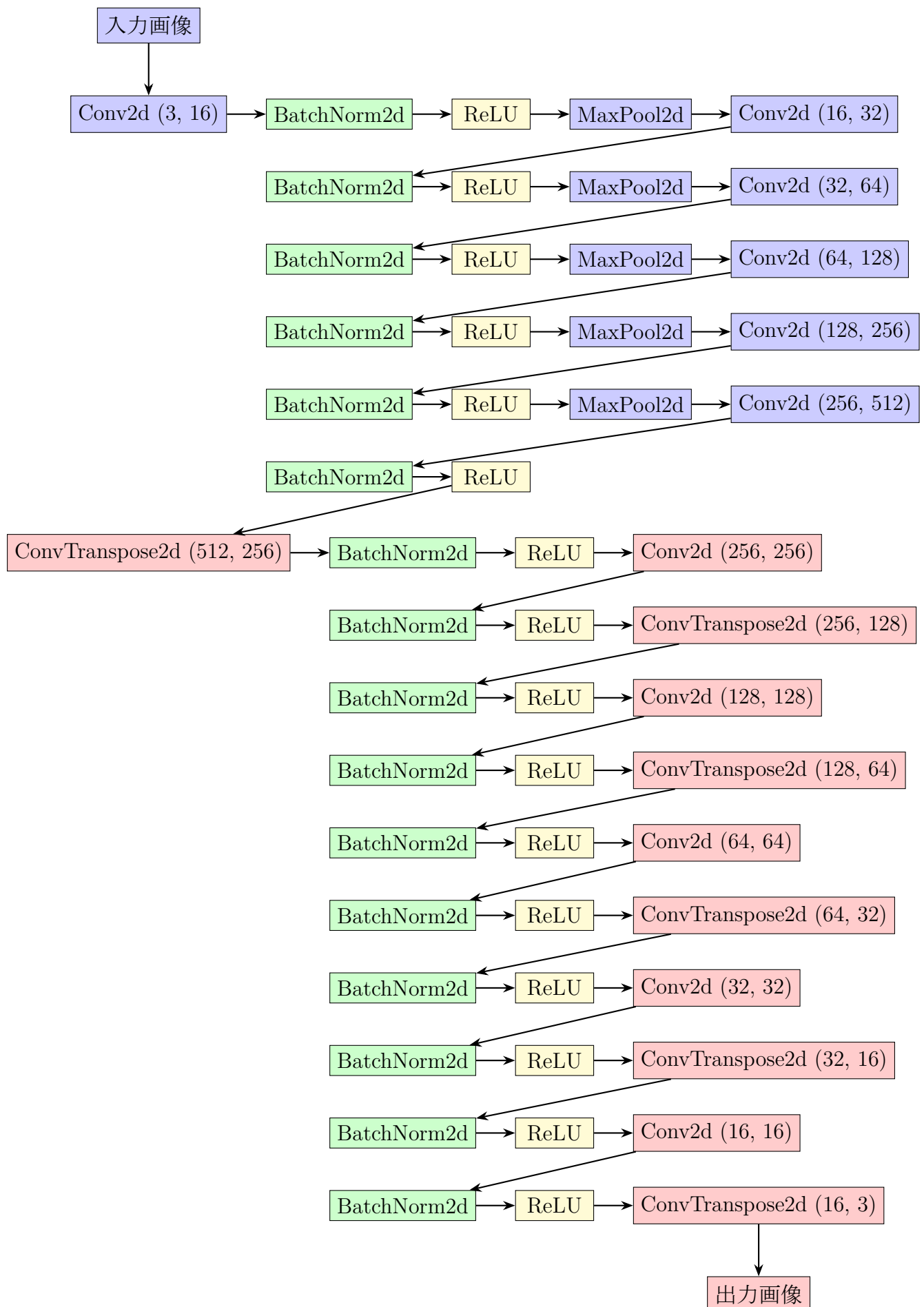


Figure 1.2: AutoEncoder の構造

現在の **AutoEncoder** モデル

現在の AutoEncoder モデルは、以前のモデルと比較して構造がシンプルになっている。以下に、現在のモデルの構造を示す。

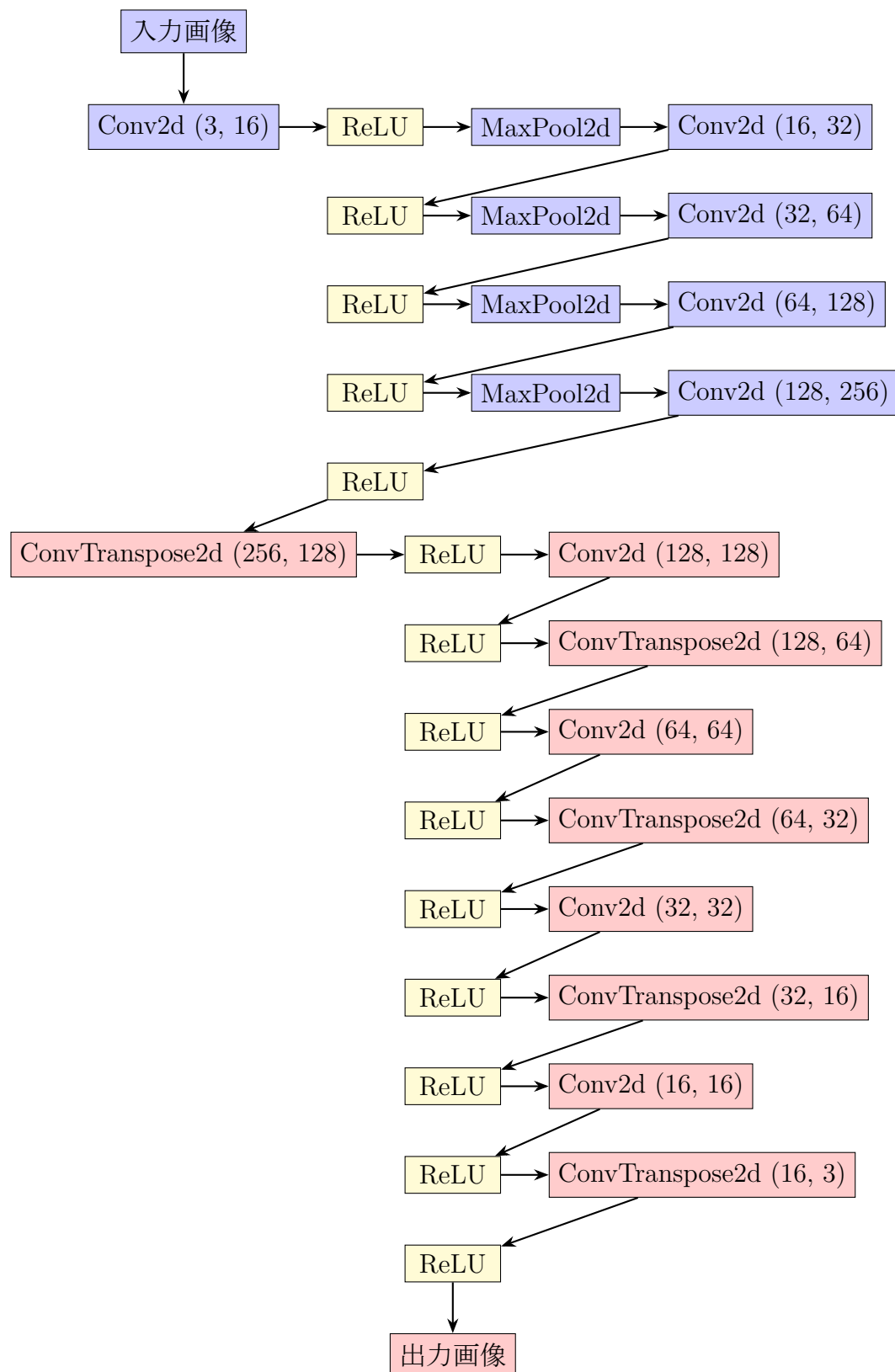


Figure 1.3: 現在の AutoEncoder の構造

変更点

- 畳み込み層と逆畳み込み層の数を減らし、モデルの深さを浅くした。
- バッチ正規化層を削除し、計算コストを削減した。
- ReLU 活性化関数の使用を最小限に抑えた。

改良の効果

これらの変更により、以下のような改善点が見られた。

- 計算コストの削減: モデルの深さを浅くし、バッチ正規化層を削除することで、計算コストが大幅に削減された。これにより、学習速度が向上し、より短時間でモデルのトレーニングが完了するようになった。
- 過学習の防止: モデルのパラメータ数を減らすことで、過学習のリスクが低減された。これにより、テストデータに対する汎化性能が向上し、より安定した結果が得られるようになった。
- モデルの解釈性の向上: 構造をシンプルにすることで、モデルの挙動がより理解しやすくなった。これにより、モデルの改善点や問題点を特定しやすくなり、さらなる改良が容易になった。

以上のように、現在の AutoEncoder モデルは、以前のモデルと比較して構造がシンプルになり、計算コストの削減、過学習の防止、モデルの解釈性の向上といった改善点が見られた。

AutoEncoder モデルのアップデートの重要性

AutoEncoder モデルをアップデートすることは、モデルの性能向上や新しいデータへの適応において非常に重要だ。以下に、モデルをアップデートしないで使い続けることと、アップデートして使うことの差について説明する。

モデルをアップデートしない場合 モデルをアップデートしないで使い続けると、以下のような問題が発生する可能性がある。

- 性能の劣化: 新しいデータや環境に適応できず、モデルの性能が徐々に劣化する可能性がある。例えば、[2] では、モデルの性能がデータの変化に対して敏感であることが示されている。
- 過学習: 古いデータに対して過学習してしまい、新しいデータに対して一般化能力が低下する可能性がある。過学習は、モデルの汎化性能を低下させる主要な要因の一つだ [3]。

モデルをアップデートする場合 一方、モデルを定期的にアップデートすることで、以下のような利点がある。

- 性能向上: 新しいデータや技術を取り入れることで、モデルの性能を向上させることができる。例えば、[1] では、ResNet のような新しいアーキテクチャを導入することで、画像認識の精度が大幅に向上したことが示されている。

- 適応能力の向上: モデルが新しいデータや環境に適応できるようになり、汎化性能が向上する。これは、モデルがより広範なデータセットに対して有効であることを意味する [4]。

1.2.4 モデルのアップデート方法

本研究では、検出に使用した画像を用いてモデルのアップデートを行う。目標は自動アップデートである。アップデート用学習データを自動で作成するためには、以下の二つのステップが必要である。

始めに、全体の画像を背景ごとにクラスタリングする。これは AutoEncoder で特徴抽出を行い、その後主成分分析で次元削減し、k-means クラスタリングを用いることで可能であることがわかった。

次に、画像の中から物体画像を取り除く。これは CLIP を使用することで効果的に行うことができる。CLIP を用いることで、画像内の物体をある程度の精度で検出し、物体が存在しない画像を抽出することができる。

これらのステップに関しては、以降の章で詳しく説明する。

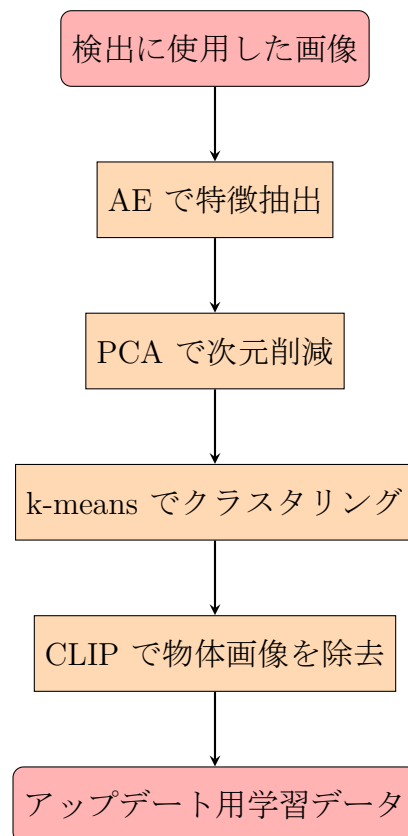


Figure 1.4: モデルのアップデート方法の流れ

1.3 使用したデータの説明

本章では、本研究で使ったデータについて説明する。まず、実験に使ったプログラミング環境とライブラリについて述べ、その後、実験に使ったファイルとディレクトリの構造を示す。さらに、使った画像データとモデルデータについて詳細に説明する。

1.3.1 プログラミング環境と使用したライブラリ

実験は Python を用いて行った。主なプログラミング環境と使用したライブラリは以下の通りである。

- **Python 3.10.12**: プログラミング言語として使用。
- **collections.defaultdict** (標準ライブラリ): デフォルト値を持つ辞書に使用。
- **csv** (標準ライブラリ): CSV ファイルの操作に使用。
- **cv2** (4.10.0.84): 画像処理に使用。
- **datetime** (標準ライブラリ): 日付と時間の操作に使用。
- **deep_translator** (1.11.4): テキストの翻訳に使用。
- **glob** (標準ライブラリ): ファイルパスのパターンマッチングに使用。
- **matplotlib.pyplot** (3.10.0): グラフの描画に使用。
- **numpy** (1.26.3): 数値計算に使用。
- **open_clip** (2.29.0): CLIP モデルの使用。
- **open_clip.tokenizer** (2.29.0): テキストのトークン化に使用。
- **os** (標準ライブラリ): OS 機能へのアクセスに使用。
- **PIL.Image** (11.0.0): 画像操作に使用。
- **PIL.ImageDraw** (11.0.0): 画像に描画するために使用。
- **PIL.ImageFont** (11.0.0): 画像にフォントを使用するために使用。
- **PIL.ImageOps** (11.0.0): 画像の操作に使用。
- **random** (標準ライブラリ): 乱数生成に使用。
- **re** (標準ライブラリ): 正規表現操作に使用。
- **shutil** (標準ライブラリ): 高レベルのファイル操作に使用。
- **sklearn.cluster.KMeans** (1.6.0): K-means クラスタリングに使用。
- **sklearn.decomposition.PCA** (1.6.0): 主成分分析に使用。
- **sklearn.preprocessing.LabelEncoder** (1.6.0): ラベルエンコーディングに使用。
- **torch** (2.5.1+cu118): 深層学習モデルの構築と訓練に使用。
- **torch.nn** (2.5.1+cu118): ニューラルネットワークの構築に使用。
- **torch.optim** (2.5.1+cu118): 最適化アルゴリズムに使用。
- **torch.utils.data.DataLoader** (2.5.1+cu118): データローダーの操作に使用。

- `torch.utils.data.Dataset` (2.5.1+cu118): データセットの操作に使用。
- `torchvision.transforms` (0.20.1+cu118): 画像の前処理に使用。
- `tqdm` (4.67.1): プログレスバーの表示に使用。

1.3.2 想定環境

```

/
2023_main.py
2024_main.py
AE_clip.ipynb
AE_deep_check.ipynb
AE_deep_check_change.ipynb
AE_deep_finetrain_matomete.py
AE_deep_main_search.ipynb
AE_deep_train_matomete.py
AE_for_newmodel.ipynb
AE_score_check_labels.ipynb
imgs/
models/

```

1.3.3 画像データ

- `imgs/for_update_img`:
 - 説明: 2024 年 9 月 3 日に撮影した 3000×4000 ピクセルの画像が 156 枚含まれている。
- `imgs/old_all_img/train_img_6048/6048/A_train_img`:
 - 説明: 去年使用した 224×224 ピクセルの画像が背景ごとに分類されずに 1512 枚ずつ、計 6048 枚含まれている。
- `imgs/make_pca_img1000`:
 - 説明: `A_train_img` と重複しない画像が 1000 枚ずつ、計 4000 枚含まれている。
- `imgs/old_train_img3/train_img_6048/6048`:
 - `new_dark/`: jpg が 6048 枚
 - `new_light/`: jpg が 6048 枚
 - `new_white/`: jpg が 6048 枚
 - 説明: 去年学習に使用した背景ごとに分類された 224×224 ピクセルの画像がそれぞれ含まれている。
- `imgs/test_img`:

- 説明: 2024 年 9 月 3 日に撮影した `for_update_img` には含まれていない 3000×4000 ピクセルの画像が 4 枚含まれている。
- `imgs/test_objects/`:
 - 説明: `test_img` を 224×224 ピクセルに分割した画像の中から、物体が存在する画像
- `imgs/test_img2/`:
 - `yes/`:
 - * `dark/`: jpg が 100 枚
 - * `light/`: jpg が 100 枚
 - * `white/`: jpg が 100 枚
 - `no/`:
 - * `dark/`: jpg が 100 枚
 - * `light/`: jpg が 100 枚
 - * `white/`: jpg が 100 枚
 - 説明: `test_img` とは別の 2024 年 9 月 3 日に撮影した `for_update_img` には含まれていない画像の中から、テスト用に背景ごとの物体が存在する画像と存在しない画像に分類したもの。
- `imgs/update_img_remove/`:
 - `dark_object/`: jpg が 306 枚
 - `dark/`: jpg が 4973 枚
 - `light_object/`: jpg が 1703 枚
 - `light/`: jpg が 7518 枚
 - `white_object/`: jpg が 1925 枚
 - `white/`: jpg が 3064 枚
 - 説明: `update_img` (今後製作される) を背景ごとに物体が存在する画像としない画像に分けたもの。

1.3.4 モデルデータ

- `models/2023models/`:
 - `AEmodel_dark_green20230927.pth`
 - `AEmodel_light_green20230927.pth`
 - `AEmodel_white20230927.pth`
 - `AEmodel_paved_ground20230927.pth`
 - 説明: 去年製作した背景ごとに 224×224 ピクセルの画像を 6048 枚学習させたモデル。

1.4 実験

本章では、物体検出を試みたうえでモデルアップデートの有用性について論じる。

1.4.1 物体検出の試み

本研究では、物体検出を行うために以下のプログラムファイルを使用した。

使用プログラム

- `2023__main.py`

使用画像

- `imgs/test_img`
- `imgs/test_objects`

使用モデル

- `models/2023models/AEmodel_dark_green20230927.pth`
- `models/2023models/AEmodel_light_green20230927.pth`
- `models/2023models/AEmodel_white20230927.pth`
- `models/2023models/AEmodel_paved_ground20230927.pth`

出力

出力

プログラムの実行結果として、`imgs/AE_2023_result` フォルダが作成され、物体だと思った画像が保存される。また、答えの画像も渡しているため、F2 スコアが計算される。

出力までの流れ

1. データの準備: 画像データを前処理し、訓練データとテストデータに分割した。
2. 画像の分割: 入力画像を 224×224 ピクセルの小さな画像に分割した。分割後の枚数は 881 枚である。
3. 物体検出の実行: 分割された画像をオートエンコーダーモデルに通し、物体が存在するかどうかを判定した。
 - オートエンコーダーモデルを使用して、入力画像を再構築する。
 - 再構築された画像と元の画像の差分を計算し、差分画像を生成する。
 - 再構築された画像と元の画像の MSE（平均二乗誤差）を計算し、MSE が最も小さいモデルの差分画像を採用する。
 - 差分画像から輪郭を抽出し、最大面積の輪郭を計算する。

- 最大面積がしきい値を超える場合、物体が存在すると判定する。
4. 結果の保存: 物体が存在すると判定された画像を保存した。
 5. 評価指標の計算: 混合行列を作成し、F2 スコアを計算した（以降で詳細を説明）。

f2 スコアの計算

AE が物体だと思った画像の枚数は 366 枚で、その中に入っていた物体画像の枚数は 25 枚である。test フォルダの画像を分割した画像が 884 枚で、その中に入っている物体画像の枚数が 50 枚である。混合行列の要素は以下の通りである。

- 混合行列の要素:
 - True Positives (TP): 25 枚
 - False Positives (FP): 341 枚
 - True Negatives (TN): 493 枚
 - False Negatives (FN): 25 枚
- 精度 (Precision) と再現率 (Recall) を計算する。

$$\text{Precision} = \frac{TP}{TP+FP} = \frac{25}{25+341} \approx 0.068$$

$$\text{Recall} = \frac{TP}{TP+FN} = \frac{25}{25+25} = 0.5$$

- F2 スコアを計算する。
 - $$F2 = \frac{(1+2^2) \cdot (\text{Precision} \cdot \text{Recall})}{(2^2 \cdot \text{Precision}) + \text{Recall}} = \frac{5 \cdot (0.068 \cdot 0.5)}{4 \cdot 0.068 + 0.5} \approx 0.221$$

これにより、従来のモデルと方法では物体検出の精度が十分でないことがわかった。そのため、モデルのアップデートを行うことが必要である。また、実際に活用することをふまえ、モデルのアップデートを自動化することが重要であると考えられる。

1.4.2 モデルアップデートの有用性

本研究では、モデルアップデートの有用性を検証するために、以下のプログラムファイルを使用した。また、これは自動でアップデートを行うという前提の上での検証であることを留意してほしい。

- **AE_clip.ipynb**: CLIP モデルを使用して画像内の物体を検出し、物体が存在しない画像を抽出するためのプログラム。
- **AE_deep_check.ipynb**: 物体画像が全く入っていないモデルと若干入っているモデルの精度を比較するためのプログラム。
- **AE_deep_check_change.ipynb**: 学習枚数を変化させたときの物体画像が全く入っていないモデルと若干入っているモデルの精度を比較するためのプログラム。
- **AE_deep_finetrain_matomete.py**: 古い画像を学習させたモデルを新しい画像でアップデートするためのプログラム。

- **AE_deep_main_search.ipynb**: 差分画像から物体画像を検出する精度向上を試みるプログラム。
- **AE_deep_train_matomete.py**: 0 からモデルを学習するプログラム。
- **AE_for_newmodel.ipynb**: 新しいモデルを学習するための画像を分類するプログラム。
- **AE_score_check_labels.ipynb**: 背景画像の自動分類を検証するプログラム。

これらのプログラムファイルを用いて、モデルの自動アップデートを行い、その有用性を検証した。

1.4.3 背景画像の分類

初めに、AE の性能を最大限発揮するために、全体画像から背景ごとに画像を分類することが重要である。これから、背景ごとに画像を分類する方法を説明する。通常、AE モデルを作成するときは画像の特徴の圧縮と再構築を行い入力画像と出力画像をなるべく一致させようとする性質を利用するために学習画像は似た画像を使う。しかし、今回は 4 種類の異なった画像を均一な枚数学習させることによって、それぞれの特徴を学習したモデルを作成する。この AE モデルで分類したい画像を圧縮し、再構築する前のベクトルを取り出し主成分分析を行い、次元削減を行う。その後、k-means クラスタリングを行い、背景ごとに画像を分類する。

使用プログラム

- **AE_deep_train_matomete.py**

使用画像

- **imgs/old_all_img/train_img_6048/6048/A_train_img**

出力モデル名

- **models/old_models6048/20241224_A_train_model.pth**

出力までの流れ

プログラムと画像を用いて、以下の流れでモデルを作成した。

1. シードの固定: 再現性を確保するために、乱数シードを固定する。
2. データの準備: 指定されたディレクトリから画像データを読み込み、学習用と評価用に分割する。
3. データセットの作成: 画像データを読み込み、PyTorch のデータセットとして準備する。
4. データローダーの作成: 学習用と評価用のデータローダーを作成する。
5. モデルの定義: 深層オートエンコーダーモデルを定義する。

6. 最適化手法と誤差関数の設定: モデルの最適化手法として Adam を使用し、誤差関数として MSELoss を設定する。
7. 学習の実行: 指定されたエポック数だけモデルの学習を行う。学習中に最も良いモデルを保存する。
8. 評価の実行: 学習後、評価用データを用いてモデルの性能を評価する。
9. 結果の保存: 学習と評価の損失をプロットし、画像として保存する。また、損失の推移を CSV ファイルに保存する。

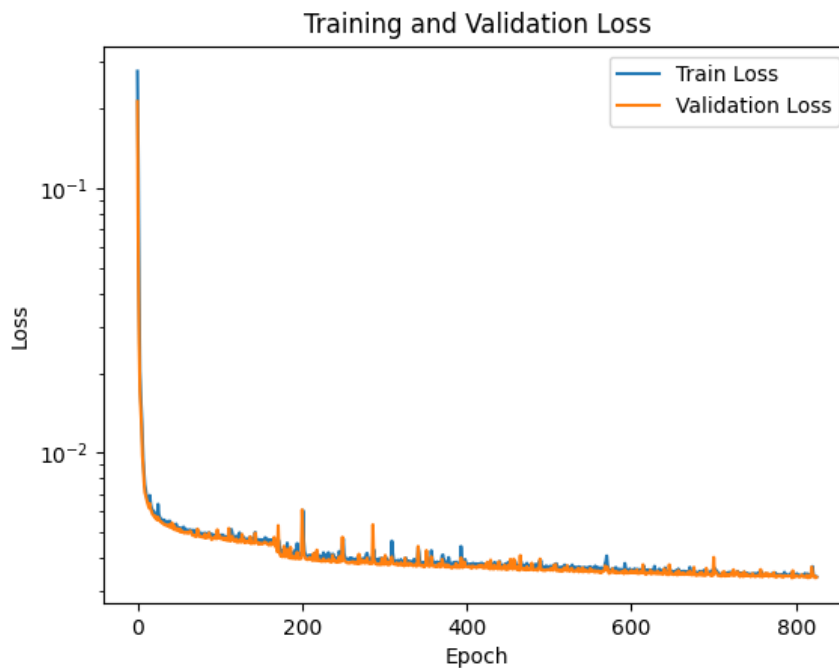


Figure 1.5: 学習推移のグラフ

使用プログラム

- `AE_score_check_labels.ipynb`

使用画像

- `imgs/make_pca_img1000`

使用モデル

- `models/old_models6048/6048_AEdeepmodel_20241224_A_train_img.pth`

出力

プログラムと画像とモデルを用いて画像の圧縮、ベクトルの主成分分析、寄与度の確認、二次元へのプロット、クラスター数 10 での k-means を行った。

出力までの流れ

1. シードの固定: 再現性を確保するために、乱数シードを固定する。
2. モデルの読み込み: 事前に訓練されたオートエンコーダーモデルを読み込む。
3. 画像の読み込み: 指定されたディレクトリから画像を読み込む。
4. オートエンコーダーによる特徴抽出: 読み込んだ画像をオートエンコーダーモデルに通し、特徴ベクトルを抽出する。
5. **PCA** による次元削減: 抽出された特徴ベクトルに対して PCA（主成分分析）を行い、次元削減を行う。
 - PCA の寄与度を確認し、主成分の重要性を評価する（以降で寄与度の詳細を説明）。
6. **PCA** 結果のプロット: PCA による次元削減結果をプロットし、視覚的に確認する（以降で PCA のプロットの詳細を説明）。
7. **K-means** クラスタリング: 次元削減された特徴ベクトルに対して K-means クラスタリングを行い、画像をクラスタに分類する。
 - クラスタ番号を再割り当てし、クラスタごとに画像を保存する。
8. クラスタリング結果のプロット: K-means クラスタリングの結果をプロットし、各クラスタの分布を視覚的に確認する（以降で k-means のプロットの詳細を説明）。

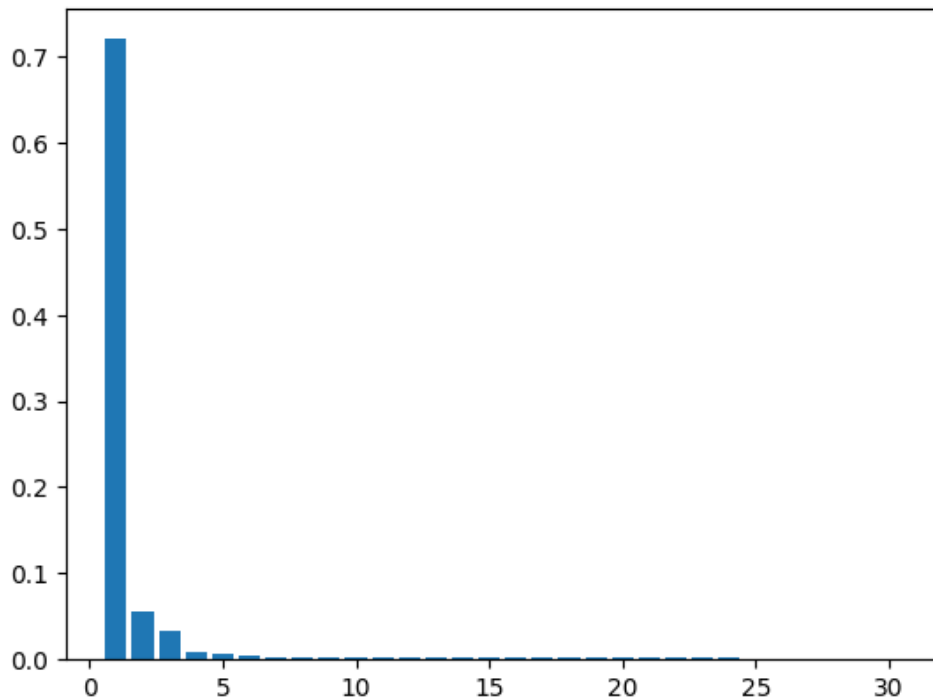


Figure 1.6: 主成分分析の寄与度

寄与度の確認 主成分分析の寄与度を示すグラフである。このグラフにより、各主成分がデータの分散にどの程度寄与しているかを確認することができる。ほとんどの寄与度が第一主成分と第二主成分で占められていることがわかる。したがって、二次元にプロットしてもデータの重要な情報を保持できることがわかる。

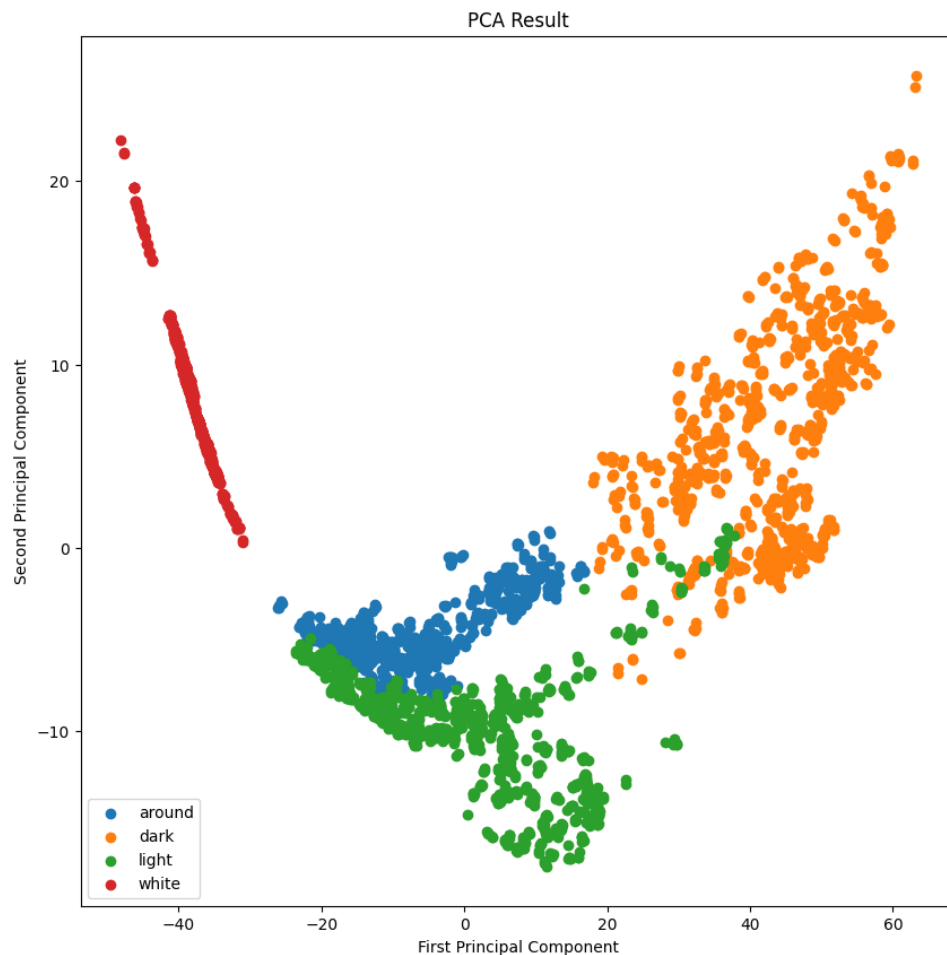


Figure 1.7: 主成分分析による二次元プロット

二次元プロット 主成分分析によって次元削減されたデータの二次元プロットである。このプロットにより、データの分布やクラスタリングの傾向を視覚的に確認することができる。結果より、around と light が近く、その後は離れていることがわかる。このことから、around と light は似た特徴を持っていることが示唆されるため、合わせて 1 種類の背景とし light と扱うことにする。そのため、以降は背景画像は 3 種類とする。

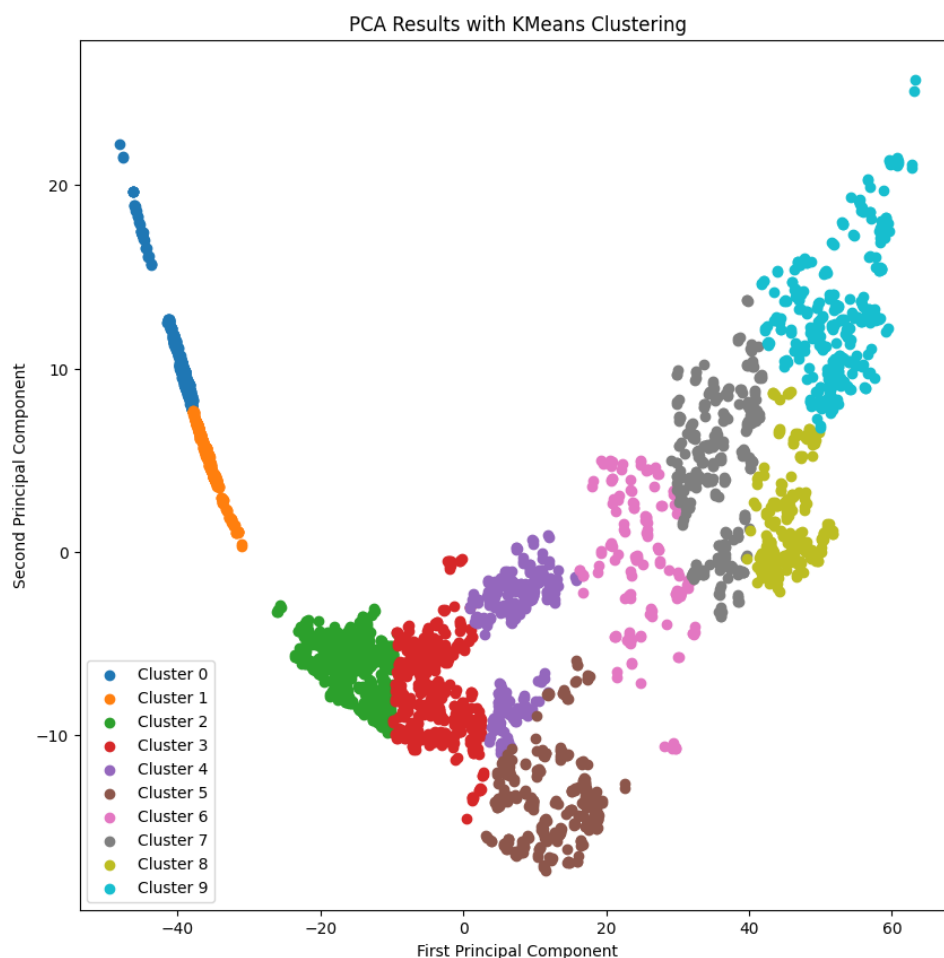


Figure 1.8: k-means クラスタリングの結果

k-means クラスタリング クラスタ数 10 の k-means クラスタリングの結果を示すプロットである。このプロットにより、データがどのようにクラスターに分割されたかを視覚的に確認することができる。横軸の値が小さいクラスターから順に数字を 0 から 9 まで振ることで結果が固定される。この結果をもとに、0 と 1 は white、4 と 5 は light、8 と 9 は dark とすることで曖昧な境界部分は除かれつつ画像を背景ごとに分類することができる。

使用プログラム

- `AE_for_newmodel.ipynb`

使用画像

- `imgs/for_update_img`

使用モデル

- `models/old_models6048/6048_AEdeepmodel_20241224_A_train_img.pth`

出力

プログラムと画像とモデルを用いて、画像の圧縮、ベクトルの主成分分析、二次元へのプロット、クラスター数 10 での k-means を行い、以下の流れで入力画像を背景ごとに分類した。この時、主成分分析に使用したモデルは `AE_score_check_labels.ipynb` で使用したモデルと同じことに注意する。

保存先

- `imgs/update_img`:
 - `dark`
 - `light`
 - `white`

出力までの流れ

1. シードの固定: 再現性を確保するために、乱数シードを固定する。
2. モデルの読み込み: 事前に訓練されたオートエンコーダーモデルを読み込む。
3. 画像の読み込み: 指定されたディレクトリから画像を読み込む。
4. 画像の分割: 読み込んだ画像を 224×224 ピクセルの小さな画像に分割する。
5. オートエンコーダーによる特徴抽出: 分割された画像をオートエンコーダーモデルに通し、特徴ベクトルを抽出する。
6. **PCA** による次元削減: 抽出された特徴ベクトルに対して PCA（主成分分析）を行い、次元削減を行う。
 - PCA の寄与度を確認し、主成分の重要性を評価する。
7. **PCA** 結果のプロット: PCA による次元削減結果をプロットし、視覚的に確認する。
8. **K-means** クラスタリング: 次元削減された特徴ベクトルに対して K-means クラスタリングを行い、画像をクラスタに分類する。
 - クラスタ番号を再割り当てし、クラスタごとに画像を保存する。
9. クラスタリング結果のプロット: K-means クラスタリングの結果をプロットし、各クラスタの分布を視覚的に確認する（以降で k-means のプロットの詳細を説明）。
10. 画像の移動: クラスタリング結果に基づいて、画像を新しいフォルダに移動する。
 - クラスタ番号に応じて、画像を 'white'、'light'、'dark' のフォルダに分類する。

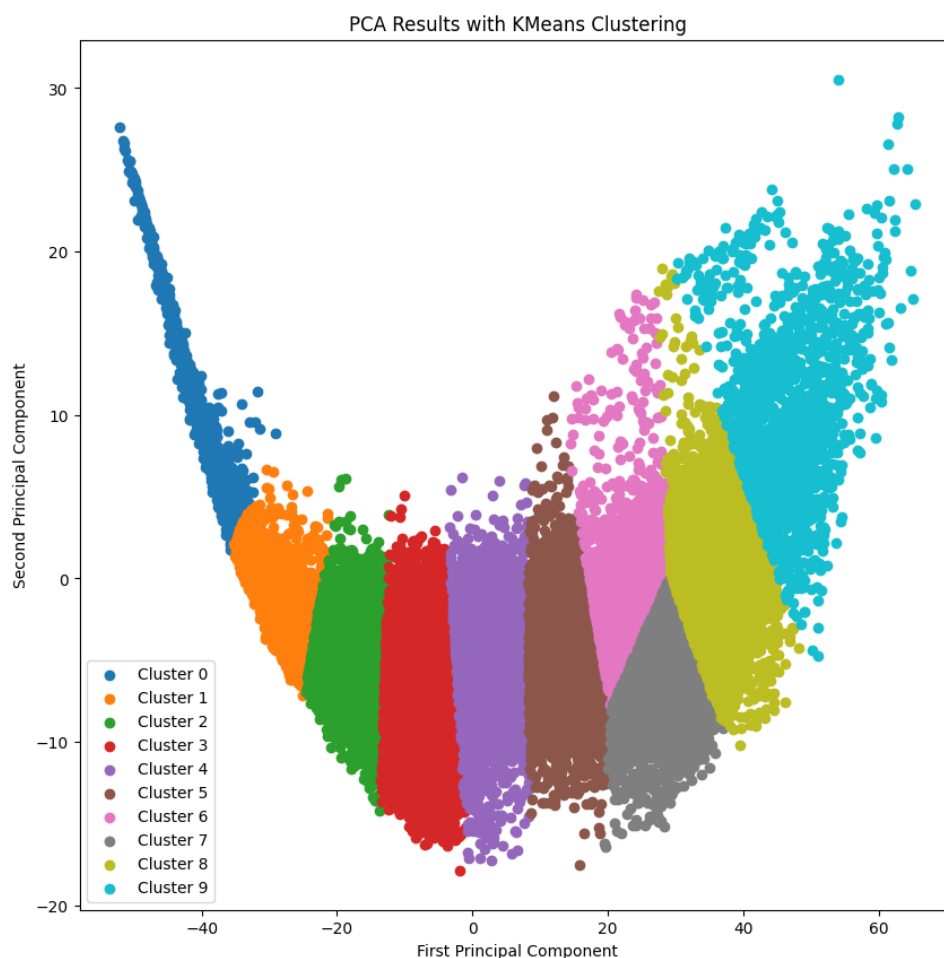


Figure 1.9: k-means クラスタリングの結果

1.4.4 物体画像の抽出

次に、背景ごとに分類された画像から物体画像を抽出する。これは、CLIP モデルを使用して画像内の物体を検出し、物体が存在する画像を抽出することで行う。

使用プログラム

- AE_clip.ipynb

使用モデル 本研究で使用したモデルは以下の通りである。

- RN50
- RN50x64

使用した言葉

本研究で使用した言葉は以下の通りである。これらの言葉は CLIP モデルに入力するために GoogleTranslator で英語に翻訳されて使用された。

- **NO_A:** 上空から緑色や黄緑色の草が生い茂った箇所を見下ろした画像で全体をくまなく探してもゴミなどの人工物、白色の小さい物体、黒色の物体、木の枝は確実に写っていない
- **NO_B:** 上空から緑色や黄緑色の草が砂浜の上から生えているのを見下ろした画像で全体をくまなく探してもゴミなどの人工物、白色の小さい物体、黒色の物体、木の枝は確実に写っていない
- **NO_C:** 上空から緑色の葉が生えた木と隙間に砂浜が見られる箇所画像で全体をくまなく探してもゴミなどの人工物、白色の小さい物体、黒色の物体、木の枝は確実に写っていない
- **NO_D:** 上空からクリーム色の砂浜の上に少し草が生えている箇所を見下ろした画像で全体をくまなく探してもゴミなどの人工物、白色の小さい物体、黒色の物体、木の枝は確実に写っていない
- **NO_E:** 上空からクリーム色の砂浜を見下ろした画像で全体をくまなく探してもゴミなどの人工物、白色の小さい物体、黒色の物体、木の枝は確実に写っていない
- **YES_A:** 上空から緑色や黄緑色の草が生い茂った箇所を見下ろした画像で全体をくまなく探した結果ゴミなどの人工物が写っている可能性がある
- **YES_B:** 上空から緑色の葉が生えた木を見下ろした画像で全体をくまなく探した結果ゴミなどの人工物が写っている可能性がある
- **YES_C:** 上空から緑色や黄緑色の草が砂浜の上から生えているのを見下ろした画像で全体をくまなく探した結果ゴミなどの人工物が写っている可能性がある
- **YES_D:** 上空から緑色の葉が生えた木と隙間に砂浜が見られる箇所画像で全体をくまなく探した結果ゴミなどの人工物が写っている可能性がある
- **YES_E:** 上空からクリーム色の砂浜の上に少し草が生えている箇所を見下ろした画像で全体をくまなく探した結果ゴミなどの人工物が写っている可能性がある
- **YES_F:** 上空からクリーム色の砂浜を見下ろした画像で全体をくまなく探した結果ゴミなどの人工物が写っている可能性がある

使用画像

- **imgs/update_img/:**
 - dark
 - light
 - white
- **imgs/update_img_remove/**

出力 プログラムとモデルと画像を用いて、物体画像を抽出した。また、予め物体画像を抽出しておくことで二つのモデルにおける精度比較も行った。100%物体画像を取り除くことは出来なかったため、物体画像を完全に取り除いた画像も保存し、それぞれの画像で学習を行いモデルの精度比較に使用する。また、物体を取り除いて学習データを作成する際に、ランダムに上下反転、左右反転、回転を行い、データ拡張を行った。保存先は以下の通りである。

保存先

- `imgs/clip_test:`
- `imgs/clip_test_result`
- `imgs/update_train_imgs_remove/train_img_6048:` 完全に物体を取り除いた画像
- `imgs/update_train_imgs_default/train_img_6048`

出力までの流れ

1. シードの固定: 再現性を確保するために、乱数シードを固定する。
2. モデルの読み込み: 事前に訓練された CLIP モデル (RN50x64 および RN50) を読み込む。
3. 翻訳: 使用する日本語の説明文を GoogleTranslator で英語に翻訳する。
4. 画像のコピー: 指定されたフォルダからランダムに画像を選択し、テスト用フォルダにコピーする。
5. テスト画像の予測とプロット: テスト用にコピーした画像に対して CLIP モデルを用いて予測を行い、結果をプロットして保存する (以降で test の結果の詳細を説明)。
6. 画像の予測: メインの画像に対して CLIP モデルを用いて予測を行う。
7. 画像の分類と保存: 予測結果に基づいて画像を分類し、結果を保存する。
8. フォルダのコピーと整理: 分類結果のフォルダをコピーし、不要なディレクトリを削除する。
9. 画像の移動: 物体画像を含むフォルダから画像を移動し、物体の割合を計算する (以降で物体の割合の詳細を説明)。
10. 画像のコピーと反転処理: 画像をコピーし、6048 枚になるように、必要に応じて反転や回転処理を行う。

テスト画像の結果

以下に、テスト画像の予測結果を示す。

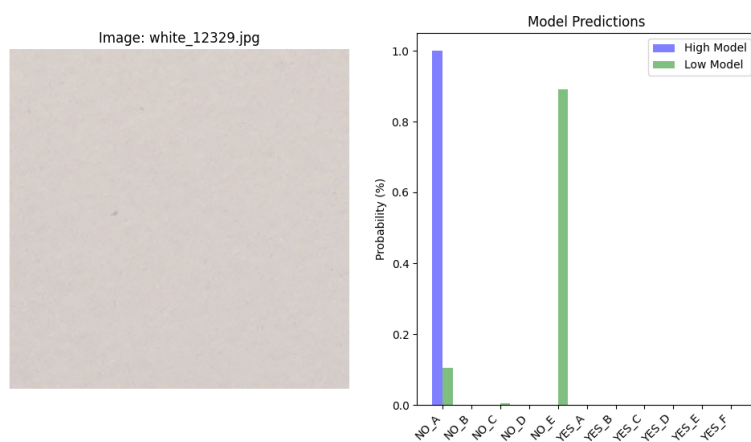


Figure 1.10: テスト画像 1 の予測結果

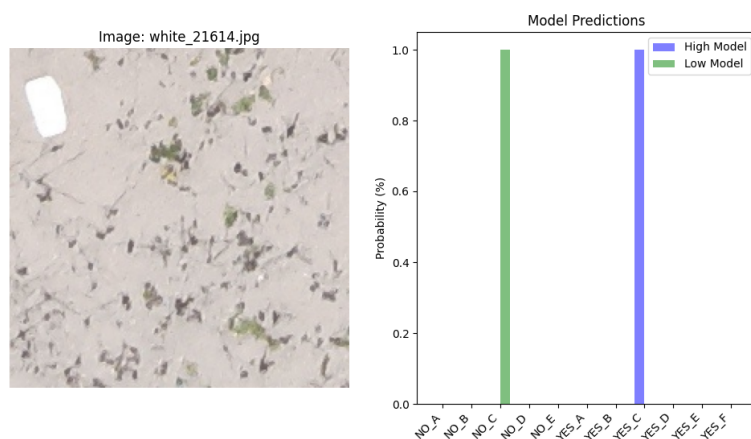


Figure 1.11: テスト画像 2 の予測結果

物体を含んでいる割合

以下に、物体を含んでいる割合を示す。

Table 1.1: RN50x64 モデルと RN50 モデルの物体を含んでいる割合の比較

カテゴリ	RN50x64	RN50
dark	1.26%	4.64%
light	11.76%	15.33%
white	20.51%	29.98%

RN50x64 モデルと **RN50** モデルの比較 RN50x64 モデルと RN50 モデルの結果を比較すると、RN50x64 モデルの方が物体を含んでいる割合が低いことがわかる。特

に、物体を含んでいる割合において、RN50x64 モデルは全てのカテゴリで RN50 モデルよりも優れている。結果から、RN50x64 モデルを使用して物体画像を抽出することが適していると言えるため以降は RN50x64 モデルを使用した画像を用いて実験を進める。なお、上記の表からもわかるように、完全に物体を取り除くことは出来なかったため、以降の章で若干物体の画像が入った画像でモデルのアップデートを行った場合と全く物体が入っていない画像でモデルのアップデートを行った場合の精度の比較を行う。

1.4.5 モデルのアップデート

次に、物体画像を含んでいる割合が低い RN50x64 モデルを使用して、モデルのアップデートを行う。モデルのアップデートは、古い画像を学習させたモデルを新しい画像でアップデートすることで行う。また、ここまでの 4 分類ではなく 3 分類の方が適していることがわかったため、アップデートをするモデルも作り直す。

旧モデルの再学習

使用プログラム

- `AE_deep_train_matomete.py`

使用画像

- `imgs/old_train_img3/train_img_6048/6048`

出力モデル名

- `models/old_models6048/20241225_new_dark.pth`
- `models/old_models6048/20241225_new_light.pth`
- `models/old_models6048/20241225_new_white.pth`

出力までの流れ

省略

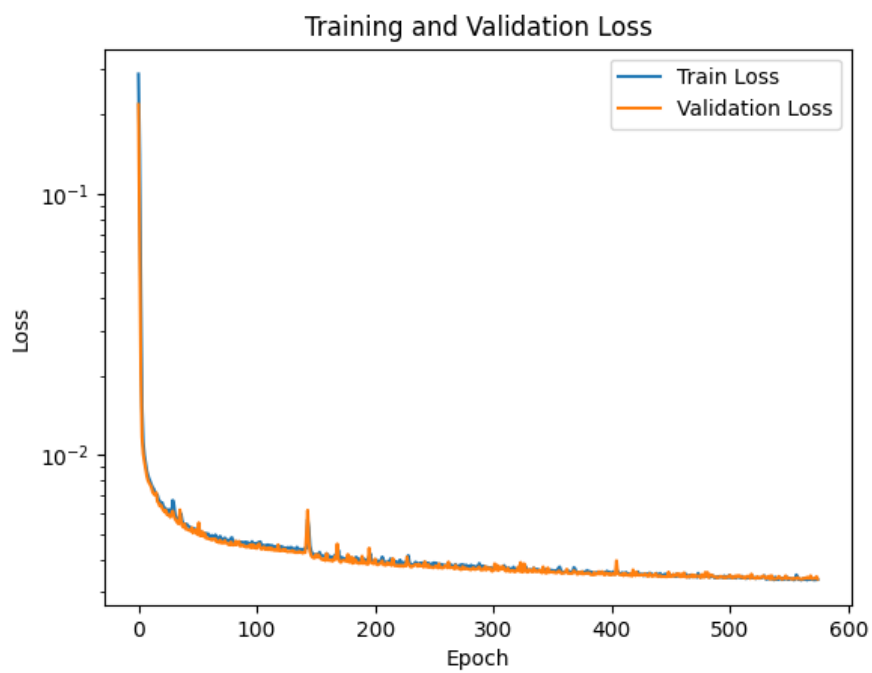


Figure 1.12: 20241225_new_dark

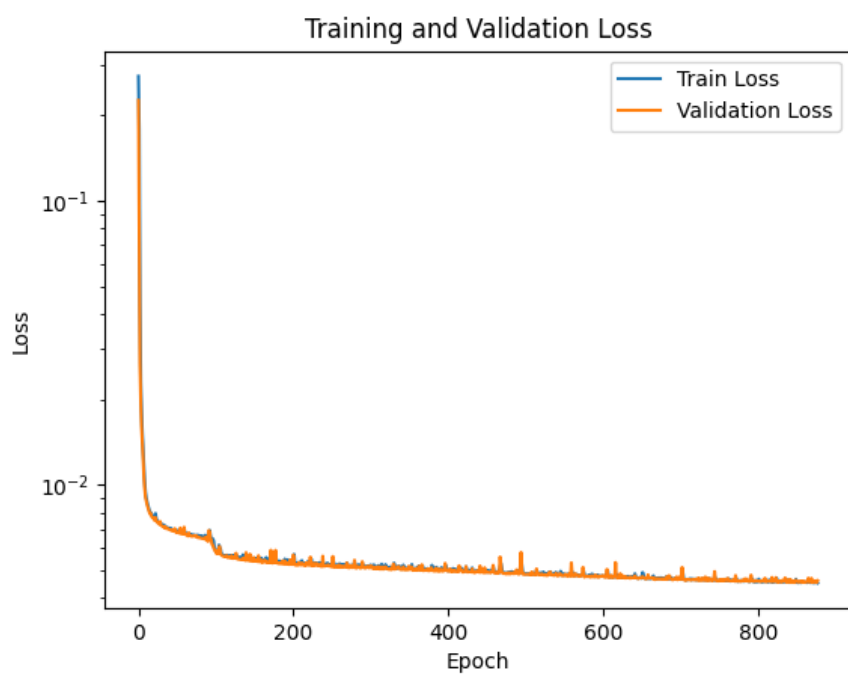


Figure 1.13: 20241225_new_light

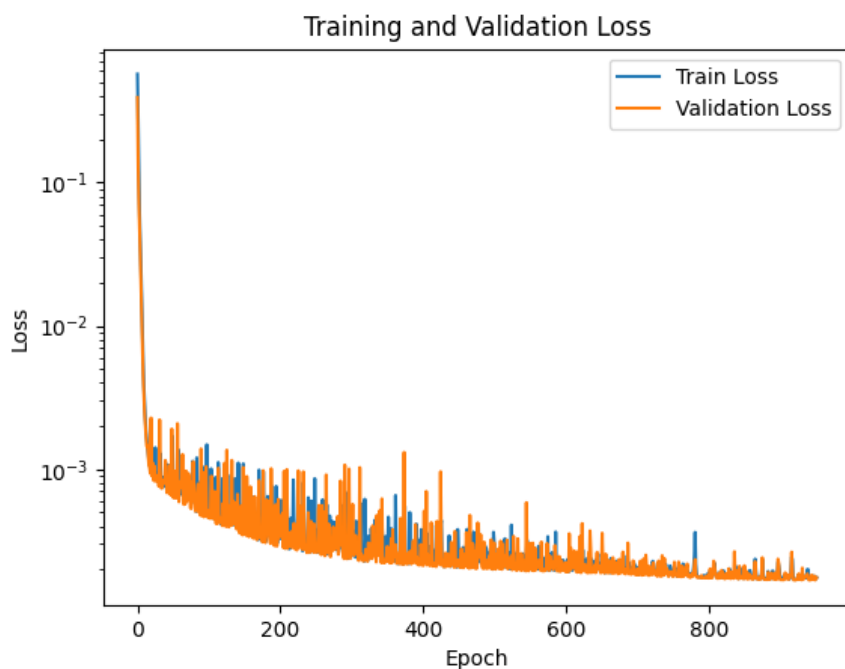


Figure 1.14: 20241225_new_white

モデルのアップデート

使用プログラム

- `AE_deep_finetrain_matomete.py`

使用画像

- `imgs/update_train_imgs_default/train_img_6048`

出力モデル名

- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_ddark.pth`
- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_dlight.pth`
- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_dwhite.pth`

出力までの流れ

1. シードの固定: 再現性を確保するために、乱数シードを固定する。
2. データの準備: 指定されたディレクトリから画像データを読み込み、学習用と評価用に分割する。
3. データセットの作成: 画像データを読み込み、PyTorch のデータセットとして準備する。

4. データローダーの作成: 学習用と評価用のデータローダーを作成する。
5. モデルの定義: 深層オートエンコーダーモデルを定義する。
6. モデルの読み込み: 事前に保存されたモデルの重みを読み込む。
7. 最適化手法と誤差関数の設定: モデルの最適化手法として Adam を使用し、誤差関数として MSELoss を設定する。
8. 学習の実行: 指定されたエポック数だけモデルの学習を行う。学習中に最も良いモデルを保存する。
9. 評価の実行: 学習後、評価用データを用いてモデルの性能を評価する。
10. 結果の保存: 学習と評価の損失をプロットし、画像として保存する。また、損失の推移を CSV ファイルに保存する。

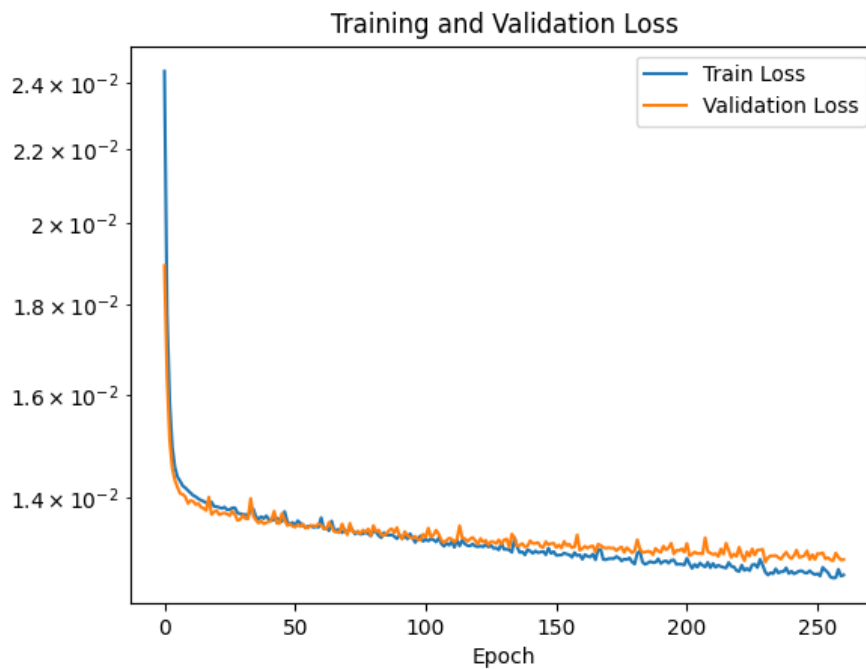


Figure 1.15: 20250115_fine6048_ddark

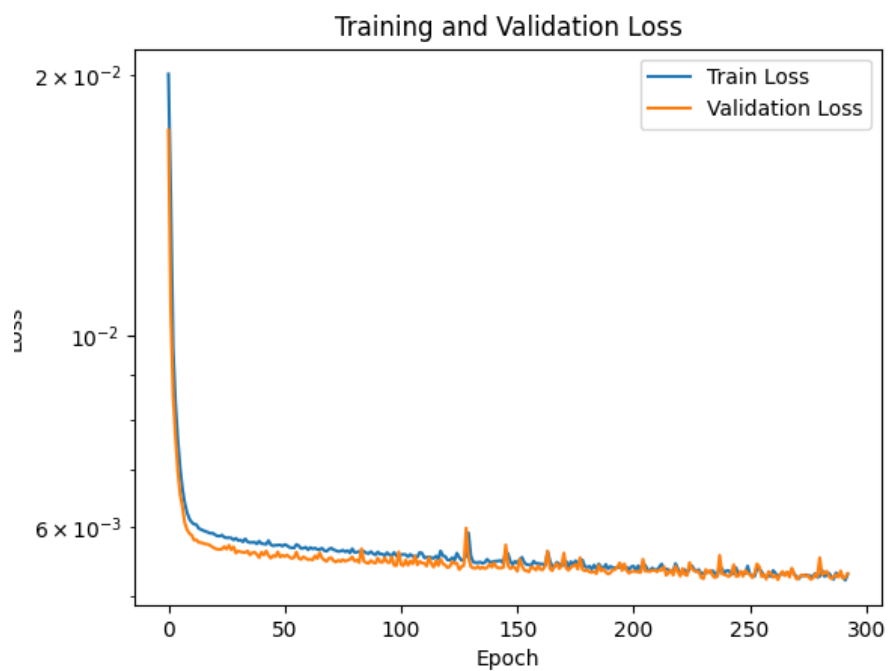


Figure 1.16: 20250115_fine6048_dlight



Figure 1.17: 20250115_fine6048_dwhite

使用プログラム

- AE_deep_finetrain_matomete.py

使用画像

- `imgs/update_train_imgs_remove/train_img_6048`

出力モデル名

- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_rdark.pth`
- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_rlight.pth`
- `models/fine_model_paths/6048_fineAEdeepmodel_20250115_rwhite.pth`

出力までの流れ

省略



Figure 1.18: 20250115_fine6048_rdark

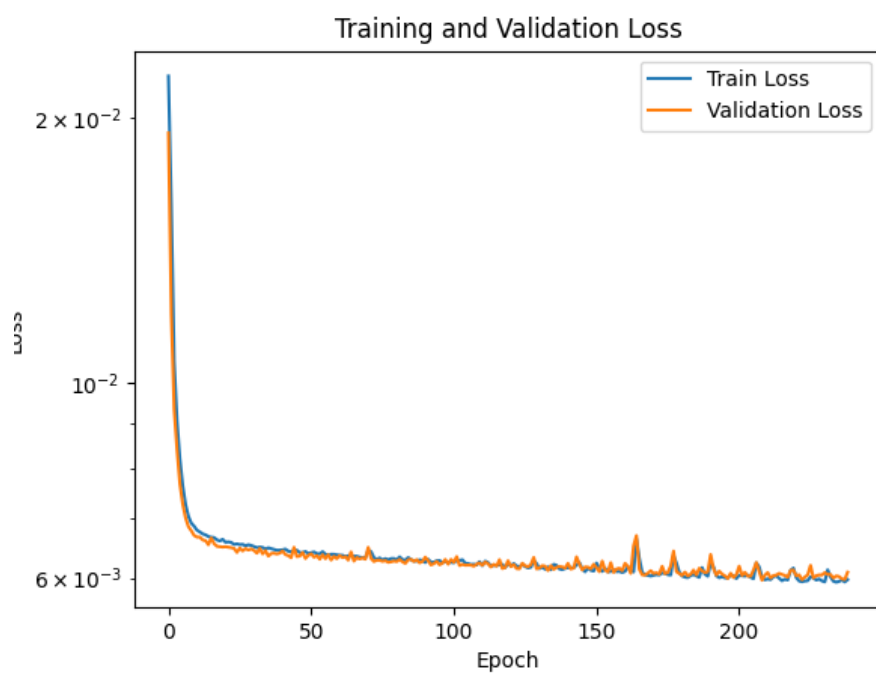


Figure 1.19: 20250115_fine6048_rlight

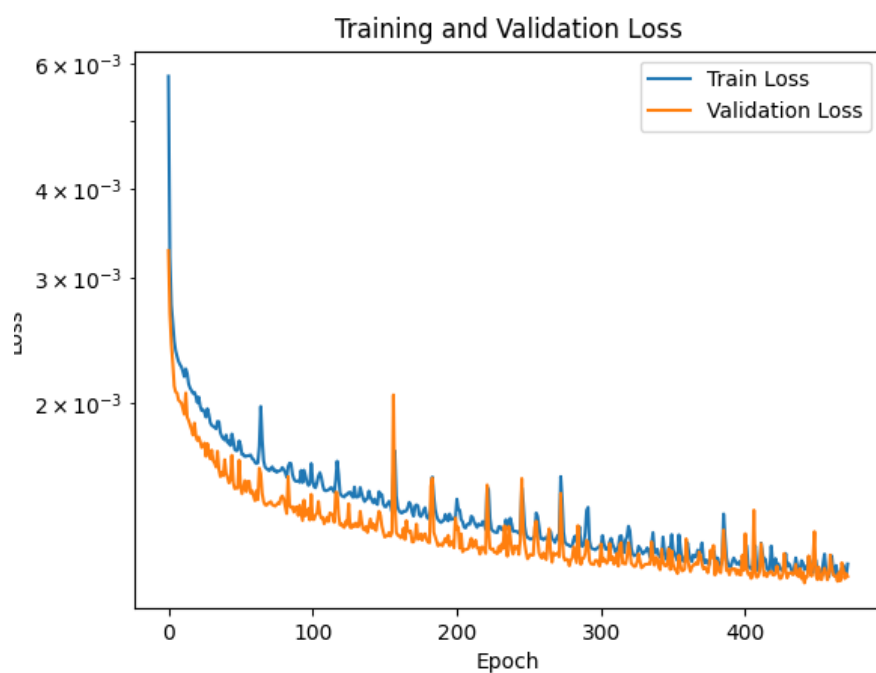


Figure 1.20: 20250115_fine6048_rwhite

1.4.6 物体検出の試み

アップデートしたモデルを使って、物体検出を行うために以下のプログラムファイルを使用した。

使用プログラム

- 2024_main.py

使用画像

- imgs/test_img
- imgs/test_objects

f2 スコアの計算

以下に、d モデルと r モデルの F2 スコアを示す。

d モデルの F2 スコア d モデルが物体だと思った画像の枚数は 278 枚で、その中に入っていた物体画像の枚数は 19 枚である。test フォルダの画像を分割した画像が 903 枚で、その中に入っていた物体画像の枚数が 50 枚である。混合行列の要素は以下の通りである。

- 混合行列の要素:
 - True Positives (TP): 19 枚
 - False Positives (FP): 259 枚
 - True Negatives (TN): 594 枚
 - False Negatives (FN): 31 枚
- 精度 (Precision) と再現率 (Recall) を計算する。

$$\text{Precision} = \frac{TP}{TP+FP} = \frac{19}{19+259} \approx 0.068$$

$$\text{Recall} = \frac{TP}{TP+FN} = \frac{19}{19+31} \approx 0.38$$

- F2 スコアを計算する。

$$\text{F2} = \frac{(1+2^2) \cdot (\text{Precision} \cdot \text{Recall})}{(2^2 \cdot \text{Precision}) + \text{Recall}} = \frac{5 \cdot (0.068 \cdot 0.38)}{4 \cdot 0.068 + 0.38} \approx 0.199$$

r モデルの F2 スコア r モデルが物体だと思った画像の枚数は 289 枚で、その中に入っていた物体画像の枚数は 22 枚である。test フォルダの画像を分割した画像が 906 枚で、その中に入っていた物体画像の枚数が 50 枚である。混合行列の要素は以下の通りである。

- 混合行列の要素:
 - True Positives (TP): 22 枚
 - False Positives (FP): 267 枚

- True Negatives (TN): 589 枚
- False Negatives (FN): 28 枚
- 精度 (Precision) と再現率 (Recall) を計算する。
 - $\text{Precision} = \frac{TP}{TP+FP} = \frac{22}{22+267} \approx 0.076$
 - $\text{Recall} = \frac{TP}{TP+FN} = \frac{22}{22+28} \approx 0.44$
- F2 スコアを計算する。

$$\text{F2} = \frac{(1+2^2) \cdot (\text{Precision} \cdot \text{Recall})}{(2^2 \cdot \text{Precision}) + \text{Recall}} = \frac{5 \cdot (0.076 \cdot 0.44)}{4 \cdot 0.076 + 0.44} \approx 0.224$$

両者ともに F2 スコアは低いが、r モデルの方が d モデルよりも F2 スコアが高いことがわかる。これは、物体画像を含んでいる割合が低いモデルを使用して学習を行ったため、物体画像を含んでいる割合が低い画像をより正確に検出できることが示唆される。しかし、あまりに低すぎるため、何故このような結果になったのかを考察する。

1.4.7 物体検出の仕組み

モデルのアップデートを行っても物体検出をすることはできなかった。ここで着目したいのは、どの段階で物体を見失っているかである。

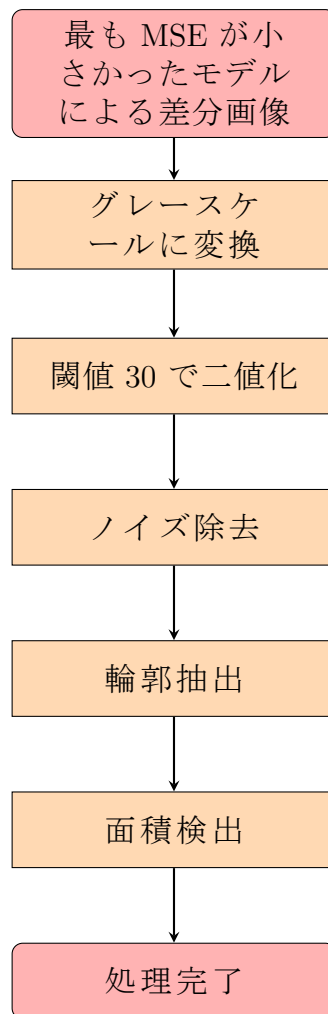


Figure 1.21: 差分画像の処理フロー

物体を検出するために、上記のステップを踏んだ。ステップ 1 つ 1 つを確認することで、物体を見失っている原因を特定する。

使用プログラム

- `AE_deep_check_tex.ipynb`

使用画像

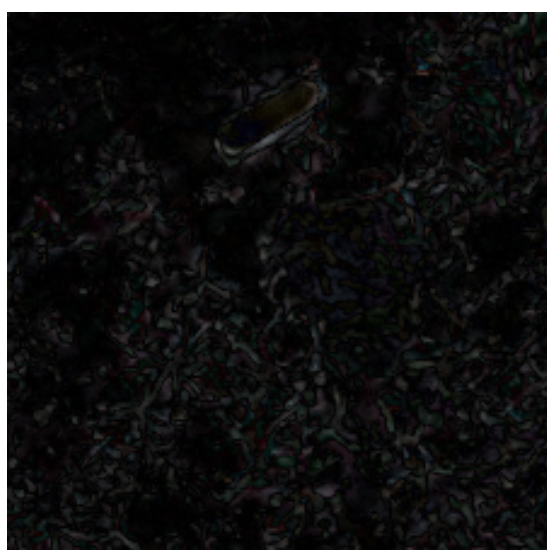
以下に、物体検出を行い両方のモデルで失敗した画像を示す。



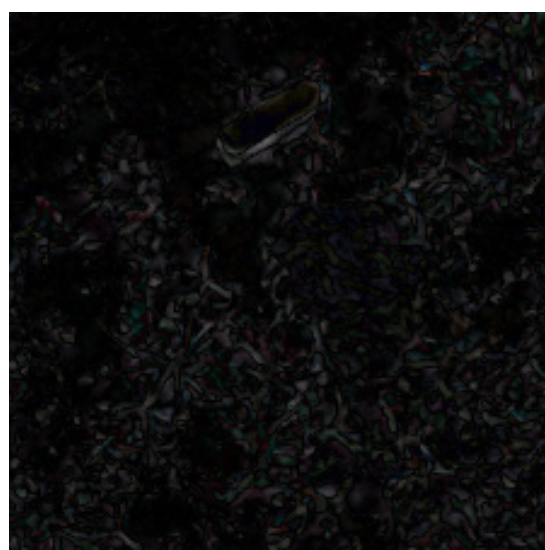
Figure 1.22: 物体検出に失敗した画像

グレースケールに変換

物体検出に失敗した画像をグレースケールに変換した結果を示す。



(a) d モデルによるグレースケール画像



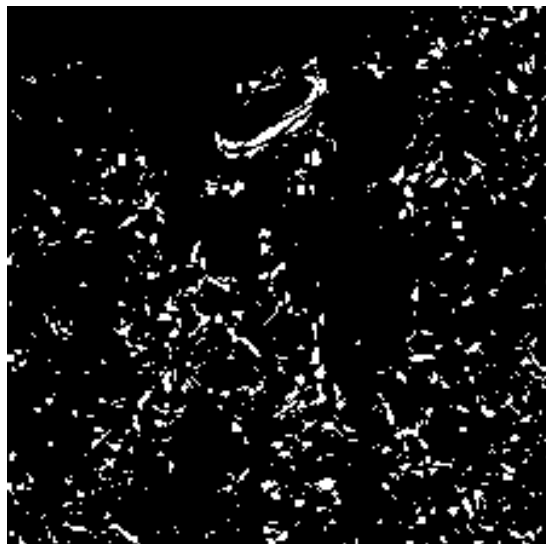
(b) r モデルによるグレースケール画像

Figure 1.23: 物体検出に失敗した画像のグレースケール変換結果

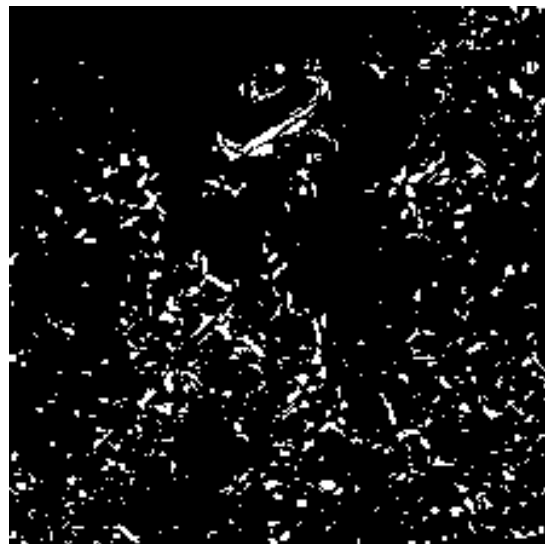
視覚的には、物体が確認できている。

二値化

次にこれらの画像を二値化する。閾値は 30 とした。



(a) d モデルによる二値化画像



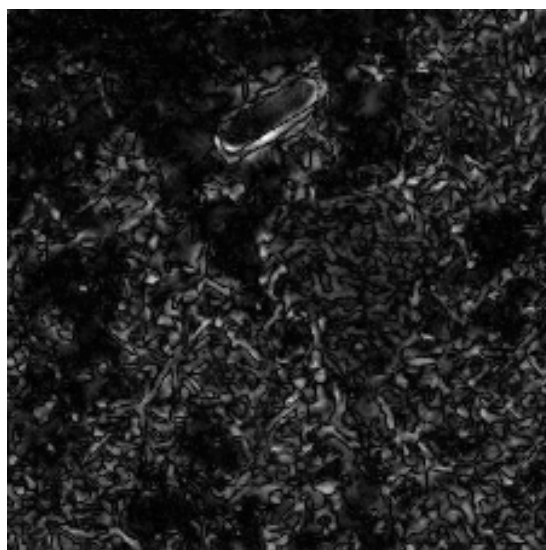
(b) r モデルによる二値化画像

Figure 1.24: 物体検出に失敗した画像の二値化結果

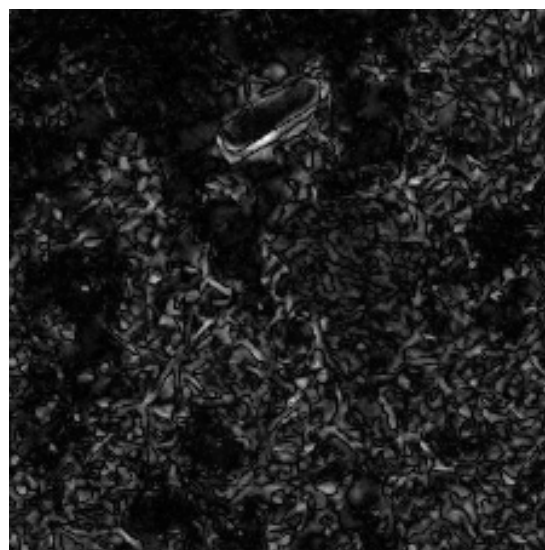
明らかに物体部分が途切れてしまっている。これを解決するために、二値化を行う前に正規化を行い、画像のコントラストを高める。

正規化をして二値化

以下に、正規化を行った後に二値化した結果を示す。閾値は 80 とした。

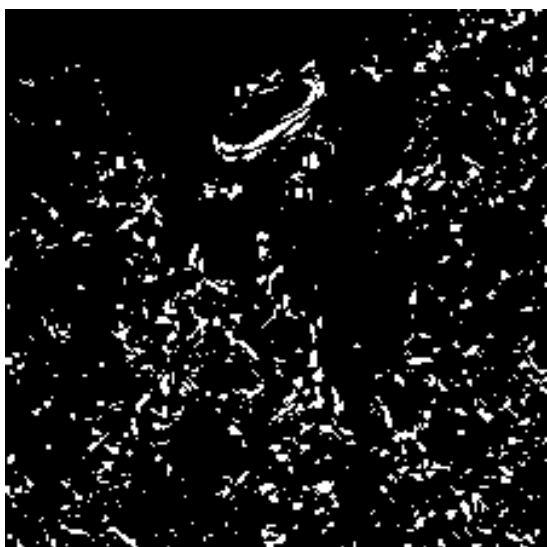


(a) d モデルによる正規化画像

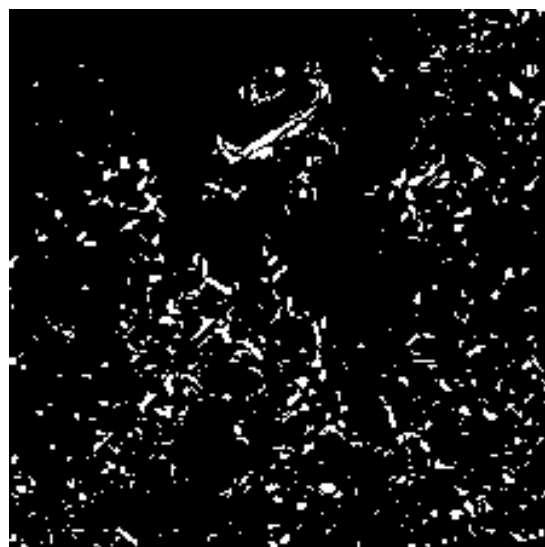


(b) r モデルによる正規化画像

Figure 1.25: 物体検出に失敗した画像の正規化



(a) d モデルによる二値化画像



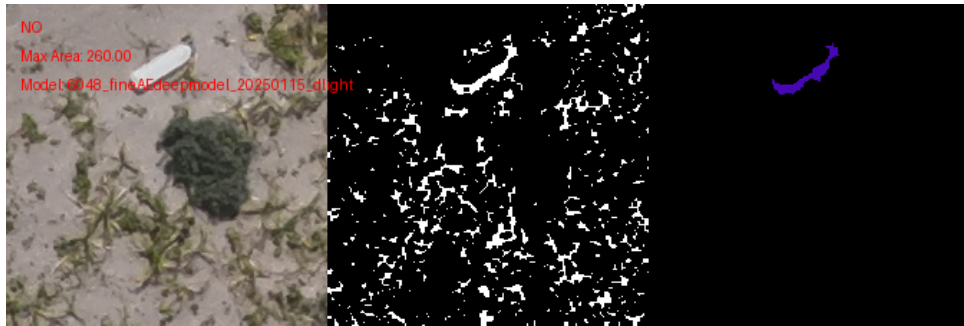
(b) r モデルによる二値化画像

Figure 1.26: 物体検出に失敗した画像の正規化と二値化

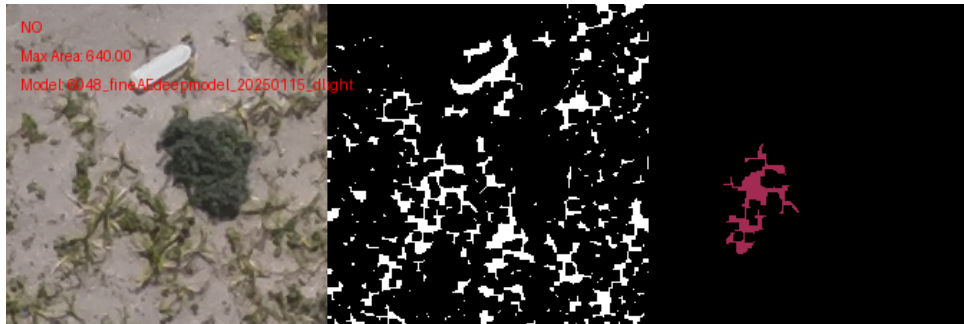
正規化を行うことで、物体部分が途切れることが改善されることはなかった。そのため、次の方法として二値化した画像に対してノイズ除去を行い、その後膨張処理と収縮処理を行うことで物体部分が途切れることの解消を目指し、物体が検出できているのかの確認を行う。

ノイズ除去及び膨張収縮

以下に、ノイズ除去及び膨張収縮を行って物体を検出した結果を示す。また、視覚的には正規化によって物体部分が途切れることが改善は見受けられなかったが、物体検出を行う際にはどのような効果が現れるかわからないのでそれも比較して検証を行う。

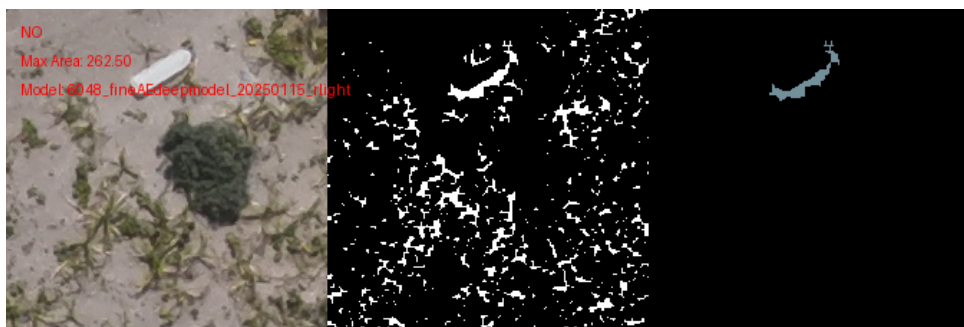


(a) d モデルによるノイズ除去後膨張収縮を行っていない画像

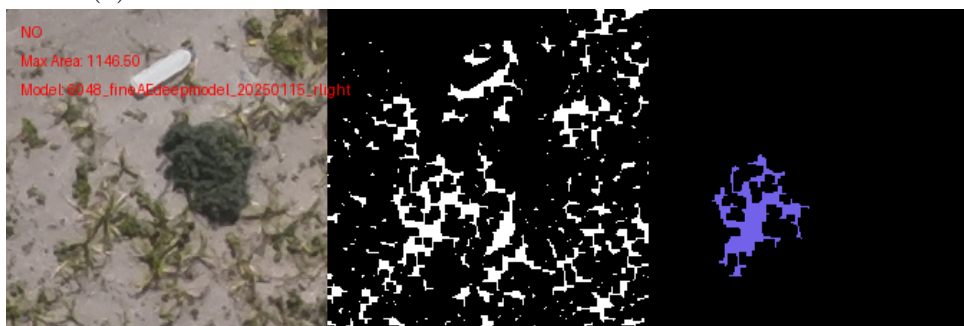


(b) d モデルによるノイズ除去及び膨張収縮画像

Figure 1.27: 物体検出に失敗した画像のノイズ除去及び膨張収縮

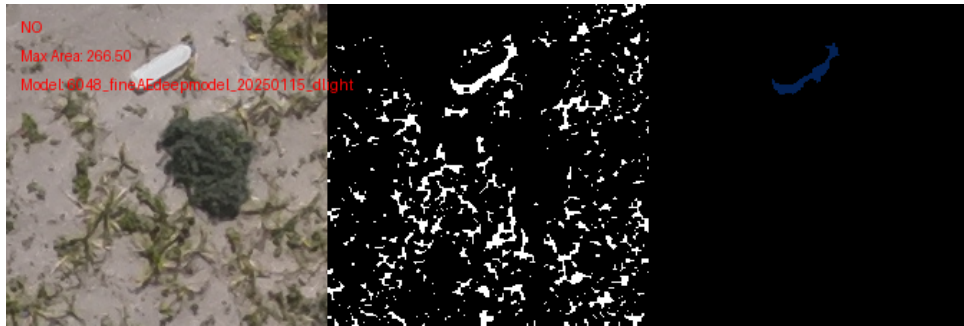


(a) r モデルによるノイズ除去後膨張収縮を行っていない画像

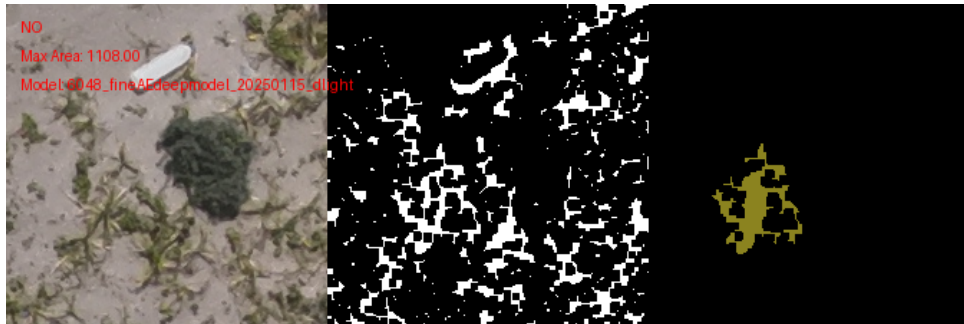


(b) r モデルによるノイズ除去及び膨張収縮画像

Figure 1.28: 物体検出に失敗した画像のノイズ除去及び膨張収縮

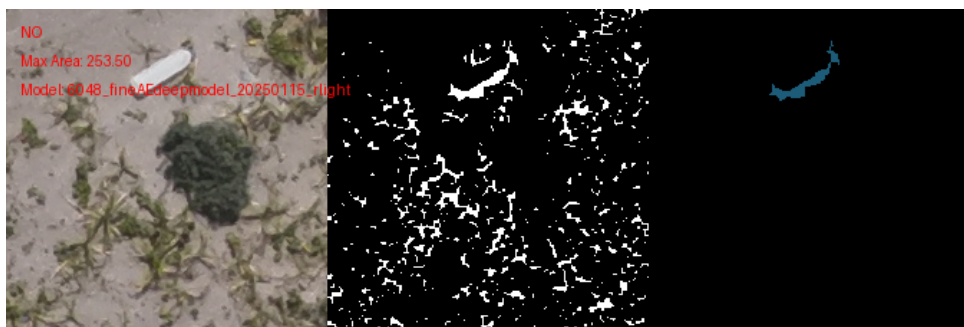


(a) d モデルによる正規化を行いノイズ除去後膨張収縮を行っていない画像

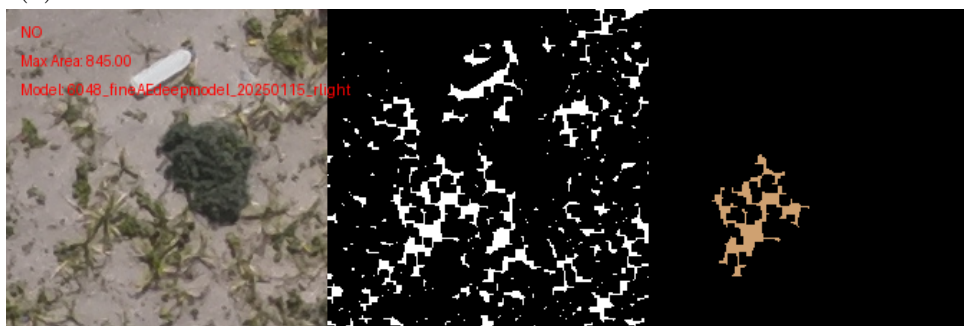


(b) d モデルによる正規化を行いノイズ除去及び膨張収縮画像

Figure 1.29: 物体検出に失敗した画像のノイズ除去及び膨張収縮



(a) r モデルによる正規化を行いノイズ除去後膨張収縮を行っていない画像



(b) r モデルによる正規化を行いノイズ除去及び膨張収縮画像

Figure 1.30: 物体検出に失敗した画像のノイズ除去及び膨張収縮

それぞれ 3 枚の画像が並んでいるが、左から順に入力画像、面積を検出する直前の画像、確認した最大面積の画像である。入力画像の左上には YESorNO と書かれており、その下には正規化を行ったかどうか、膨張収縮を行ったかどうか書かれて

いる。また、面積を検出する直前の画像の左上には YESorNO と書かれており、その下には検出した最大面積の大きさ、一番下には mse の値が一番小さくなったモデルの名前が書かれている。最大面積が閾値（今回は 1200）を超えると YES と表示される。全ての画像に共通して、物体部分の面積を適切に検出できていないことが確認できる。特に膨張収縮を行った画像に関しては物体ではない部分の面積を検出してしまっている。これは AE が再現した画像の再現性が低い部分を膨張収縮により繋げてしまったためである。また、正規化を行っても結果が改善されていないことから、正規化は必要ないことがわかる。以上の結果から、差分画像に対してアプローチを行うのではなく、出力画像の精度向上にアプローチを行うことがより良い結果を得られると考えられる。

1.4.8 出力画像の精度向上

出力画像の精度を向上させる一般的な方法は学習枚数を増やすことである。学習枚数を増やすことで、モデルがより多くのパターンを学習することができるため、出力画像の精度が向上すると考えられる。しかし今回は現状が最大枚数なため、学習枚数を減らしたモデルを作成し、そのモデルを用いて MSE の推移を確認する。また、推移を確認する画像として物体検出に失敗した画像と紹介してきた画像を使用する。

学習枚数の変更

学習枚数を減らしたデータセットを作成する。今回は 100 枚、1000 枚、2000 枚、3000 枚、4000 枚、5000 枚のデータセットを作成する。

使用プログラム

- AE_train_img_move.ipynb

使用画像

- imgs/update_train_imgs_default/train_img_6048
- imgs/update_train_imgs_remove/train_img_6048

出力

- imgs/update_train_imgs_default_mix
- imgs/update_train_imgs_remove_mix

モデルの作成

以下に、学習枚数を減らしたモデルを作成する。

使用プログラム

- AE_deep_finetrain_matomete.py

使用画像

- imgs/update_train_imgs_default_mix/train_img_100
- imgs/update_train_imgs_remove_mix/train_img_100
- imgs/update_train_imgs_default_mix/train_img_1000
- imgs/update_train_imgs_remove_mix/train_img_1000
- imgs/update_train_imgs_default_mix/train_img_2000
- imgs/update_train_imgs_remove_mix/train_img_2000
- imgs/update_train_imgs_default_mix/train_img_3000
- imgs/update_train_imgs_remove_mix/train_img_3000
- imgs/update_train_imgs_default_mix/train_img_4000
- imgs/update_train_imgs_remove_mix/train_img_4000
- imgs/update_train_imgs_default_mix/train_img_5000
- imgs/update_train_imgs_remove_mix/train_img_5000

出力モデル名

- models/fine_model_paths/100_fineAEdeepmodel_20250116_ddark.pth
- models/fine_model_paths/100_fineAEdeepmodel_20250116_dlight.pth
- models/fine_model_paths/100_fineAEdeepmodel_20250116_dwhite.pth
- models/fine_model_paths/100_fineAEdeepmodel_20250116_rdark.pth
- models/fine_model_paths/100_fineAEdeepmodel_20250116_rlight.pth
- models/fine_model_paths/100_fineAEdeepmodel_20250116_rwhite.pth
- models/fine_model_paths/1000_fineAEdeepmodel_20250116_ddark.pth
- models/fine_model_paths/1000_fineAEdeepmodel_20250116_dlight.pth
- models/fine_model_paths/1000_fineAEdeepmodel_20250116_dwhite.pth
- models/fine_model_paths/1000_fineAEdeepmodel_20250116_rdark.pth
- models/fine_model_paths/1000_fineAEdeepmodel_20250116_rlight.pth
- models/fine_model_paths/1000_fineAEdeepmodel_20250116_rwhite.pth
- models/fine_model_paths/2000_fineAEdeepmodel_20250116_ddark.pth
- models/fine_model_paths/2000_fineAEdeepmodel_20250116_dlight.pth
- models/fine_model_paths/2000_fineAEdeepmodel_20250116_dwhite.pth

- models/fine_model_paths/2000_fineAEdeepmodel_20250116_rdark.pth
- models/fine_model_paths/2000_fineAEdeepmodel_20250116_rlight.pth
- models/fine_model_paths/2000_fineAEdeepmodel_20250116_rwhite.pth
- models/fine_model_paths/3000_fineAEdeepmodel_20250116_ddark.pth
- models/fine_model_paths/3000_fineAEdeepmodel_20250116_dlight.pth
- models/fine_model_paths/3000_fineAEdeepmodel_20250116_dwhite.pth
- models/fine_model_paths/3000_fineAEdeepmodel_20250116_rdark.pth
- models/fine_model_paths/3000_fineAEdeepmodel_20250116_rlight.pth
- models/fine_model_paths/3000_fineAEdeepmodel_20250116_rwhite.pth
- models/fine_model_paths/4000_fineAEdeepmodel_20250116_ddark.pth
- models/fine_model_paths/4000_fineAEdeepmodel_20250116_dlight.pth
- models/fine_model_paths/4000_fineAEdeepmodel_20250116_dwhite.pth
- models/fine_model_paths/4000_fineAEdeepmodel_20250116_rdark.pth
- models/fine_model_paths/4000_fineAEdeepmodel_20250116_rlight.pth
- models/fine_model_paths/4000_fineAEdeepmodel_20250116_rwhite.pth
- models/fine_model_paths/5000_fineAEdeepmodel_20250116_ddark.pth
- models/fine_model_paths/5000_fineAEdeepmodel_20250116_dlight.pth
- models/fine_model_paths/5000_fineAEdeepmodel_20250116_dwhite.pth
- models/fine_model_paths/5000_fineAEdeepmodel_20250116_rdark.pth
- models/fine_model_paths/5000_fineAEdeepmodel_20250116_rlight.pth
- models/fine_model_paths/5000_fineAEdeepmodel_20250116_rwhite.pth

MSE の推移の確認

以下に、学習枚数を減らしたモデルによる物体画像の MSE の推移を確認する。

使用プログラム

- AE_deep_check_suii.ipynb

使用モデル

- models/dmodels/100_fineAEdeepmodel_20250116_dlight.pth
- models/rmodels/100_fineAEdeepmodel_20250116_rlight.pth
- models/dmodels/1000_fineAEdeepmodel_20250116_dlight.pth
- models/rmodels/1000_fineAEdeepmodel_20250116_rlight.pth
- models/dmodels/2000_fineAEdeepmodel_20250116_dlight.pth
- models/rmodels/2000_fineAEdeepmodel_20250116_rlight.pth
- models/dmodels/3000_fineAEdeepmodel_20250116_dlight.pth
- models/rmodels/3000_fineAEdeepmodel_20250116_rlight.pth
- models/dmodels/4000_fineAEdeepmodel_20250116_dlight.pth
- models/rmodels/4000_fineAEdeepmodel_20250116_rlight.pth
- models/dmodels/5000_fineAEdeepmodel_20250116_dlight.pth
- models/rmodels/5000_fineAEdeepmodel_20250116_rlight.pth
- models/dmodels/6048_fineAEdeepmodel_20250115_dlight.pth
- models/rmodels/6048_fineAEdeepmodel_20250115_rlight.pth

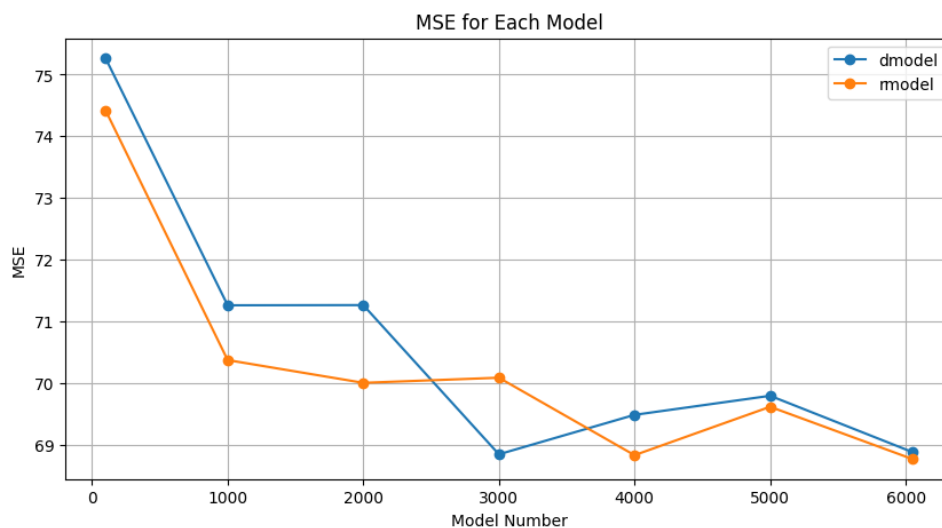


Figure 1.31: MSE の推移

出力

Bibliography

- [1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [3] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [4] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521:436–444, 2015.